Web Testing Assistant

User and technical documentation

A Windows desktop app in which Claude Code designs test cases from your product backlogs and tests your web application in a built-in browser, step by step, while you review every result before it counts.

Version 1.1.4 (pre-release) · **Date** October 2026 · **Platform** Windows 10 and 11

Source github.com/Mohamed552811/Web-Testing-Assistant

All screenshots in this document were taken from the app itself: real test runs against the local test pages that ship with the project, and a real test case generation from a small demo workbook.

Contents

1 Introduction 4

1.1 Who it is for 4

1.2 What it can do 4

1.3 How to read this document 5

2 How it works 6

2.1 Three modes 6

2.2 The parts 6

2.3 The testing workflow 7

2.4 What happens during a run 7

2.5 Statuses 7

3 Installing 9

3.1 Requirements 9

3.2 Install with Setup.exe 9

3.3 First start: the setup check 9

3.4 Updating and uninstalling 10

4 Quick start 11

5 The Launchpad and the main window 12

5.1 The Launchpad and modes 12

5.2 Parts of the window 13

5.3 Resizing, rearranging and expanding 13

5.4 Menus and toolbar 14

5.5 Plan usage and notifications 14

6 Test Case Design 16

6.1 Workbooks, mapping and product context 16

6.2 Pick a backlog and check what is sent 17

6.3 Generate, review and edit 18

6.4 Export and hand over 18

6.5 Test Case Design settings 19

7 Sessions 20

7.1 Test accounts and roles 20

7.2 File uploads 20

8 Scenarios and smoke tests 21

8.1 Scenarios 21

8.2 Smoke tests 22

8.3 The library and importing test cases 22

8.4 Let Claude suggest scenarios 24

9 Running tests 25

9.1 The live run 25

9.2 Staying in control 26

9.3 New tabs, pop-ups and printing 26

10 Reviewing results 27

10.1 The review panel 27

10.2 Deciding 28

10.3 Site notes 28

11 Manual checklist runs 29

12 Runs, history and comparison 30

12.1 Errors and speed 30

12.2 Compare with the previous run 31

13 Reports 32

14 Tools 33

14.1 Search everything 33

14.2 Trends 33

14.3 Clean up and back up 34

15 Settings 35

16 Security and privacy 36

17 Plan usage and cost control 37

18 Data and files 38

19 Troubleshooting 39

20 For developers 40

20.1 Running from source 40

20.2 Code map 40

20.3 Tests and test pages 40

20.4 Building the installer and this document 41

20.5 Versions and releases 41

A Keyboard shortcuts 42

B Glossary 42

1 Introduction

Web Testing Assistant lets an AI tester do the clicking while you stay the judge.

The app has three modes, chosen on its start screen, the **Launchpad**: **Test Case Design** turns the backlogs of an Excel workbook into test cases; **Regression Testing** and **Smoke Testing** run tests in the built-in browser. Test cases from the designer can be handed over to either testing mode.

You describe a test in plain language: the steps to follow and the result you expect. Claude Code opens your web application in the app's own browser, follows the steps, checks what happens and records every step with a status, its reasoning, and, when something is wrong, a screenshot and a ready-to-file bug report. Nothing is final until you review it: you approve, override or send each step back for a re-run. Finished sessions export to Excel, Word, PDF or Markdown.

1.1 Who it is for

- **Manual and exploratory testers** who want to hand repetitive checks to an assistant and keep full control of the verdict.
- **QA leads** who need consistent evidence: the same steps, screenshots and bug fields on every run, ready for Azure DevOps.
- **Developers** who want a quick smoke test of a page or a regression check of a feature on each new build.

1.2 What it can do

Area	Capabilities
Test case design	Upload an Excel workbook of product backlogs; Claude writes happy-path and corner-case test cases per backlog from its text and screenshots, with open questions; edit, lock, regenerate, export to JSON, CSV or Excel, or hand them over.
Checklists	Manual checklist runs from designed test cases, with Pass, Fail, Blocked and Skipped, notes, screenshots and export.
Testing	Scenarios written as numbered steps with an expected result; smoke tests that check a whole page; several test accounts (roles) per session; file uploads; new tabs and print views; accessibility checks.
Watching	A live timeline of every action and what Claude saw, with progress and a timer; a banner over the browser while Claude is in control; a Stop button that ends everything at once.
Reviewing	Status badges, filters, step cards, screenshots with the failing element marked, editable bug reports, keyboard shortcuts, site notes for the next run.
Evidence	Console errors, failed requests and page load times recorded by the app on every step; reports in Excel (Azure DevOps column names), Word, PDF and Markdown.
Regression	A scenario library, import from Excel/CSV and Azure DevOps exports, test data variables, run history and comparison with the previous run of the same test, trends per site.

Cost and safety	Runs on your Claude subscription (no API key); usage estimate and automatic pause; model per run type; allowed-domain guard; page content treated as data; test passwords kept out of every file the app writes.

1.3 How to read this document

Chapter 2 explains how the app works. Chapters 3 and 4 get you installed and through a first test in about ten minutes. Chapter 5 introduces the Launchpad and the window, chapter 6 covers Test Case Design, and chapters 7 to 15 describe testing in the order you use it. Then come security, plan usage, data and troubleshooting, and chapter 20 is for developers who build or change the app.

2 How it works

Three programs work together on your computer: the app, Claude Code and Playwright MCP.

2.1 Three modes

Mode	What Claude does	What you do
Test Case Design	Reads one backlog (text and screenshots) and writes test cases and open questions. No browser, no tools.	Review, edit and hand the test cases over.
Regression Testing	Runs your scenarios step by step in the built-in browser through Playwright MCP.	Review every step; run checklists.
Smoke Testing	Writes and runs a broad checklist for one page at a time.	Review every check; run checklists.

2.2 The parts

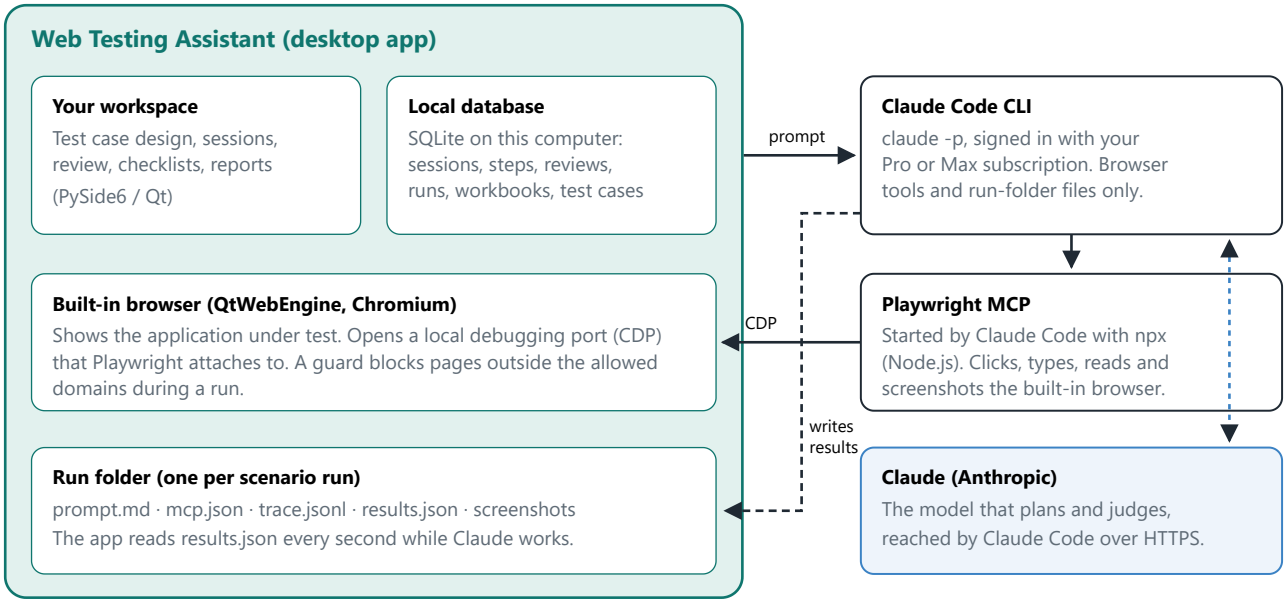


Figure 1. Everything except the Claude model runs on your computer. The app starts Claude Code for each scenario; Claude Code drives the built-in browser through Playwright MCP and writes its results into the run folder. For Test Case Design the app starts Claude Code once per backlog, without any tools.

The app is where you work: it holds your sessions and results, shows the browser and the live run, and turns results into reports. **Claude Code** is Anthropic's command-line agent. The app starts it once per scenario (`claude -p`) and gives it a prompt that describes the test, the rules and the result format. **Playwright MCP** is the browser toolkit Claude Code uses: it attaches to the app's own browser over its local debugging port, so the page you see is the page Claude tests.

> **Your subscription, not an API key**
> Claude Code runs with your own Claude Pro or Max sign-in. The app removes `ANTHROPIC_API_KEY` from the environment of every Claude process, never reads or stores Claude credentials, and shows plan usage only from what Claude Code itself reports.

2.3 The testing workflow

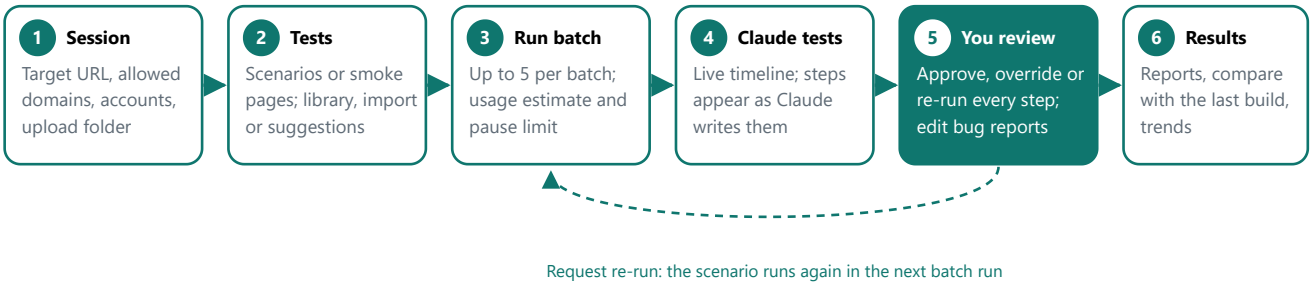


Figure 2. The testing workflow (Regression and Smoke). A session holds batches of tests. Claude runs a batch; a step is final only after your review.

2.4 What happens during a run

1. **The app prepares a run folder** for the scenario: the prompt (`prompt.md`, with the test password replaced), the Playwright MCP configuration (`mcp.json`) and, if the session has an upload folder, a copy of its files.
2. **It starts Claude Code** in that folder with a restricted set of tools: the Playwright browser tools and file reading and writing inside the run folder. Claude Code gets no shell and does not load your own Claude Code settings or hooks.
3. **Claude tests in the visible browser.** It takes snapshots of the page, clicks, types and waits, exactly like a tester. Every action, every note and everything Claude saw streams into the *Live run* tab and into `trace.jsonl`.
4. **After every step Claude rewrites** `results.json` with all steps so far. The app reads the file every second, so steps, statuses, screenshots and bug reports appear on the Scenarios tab while the run is still going.
5. **The app watches the page itself** over its own read-only debugging connection: console errors, failed requests and page load times are recorded for every step, at no cost to your plan.
6. **When the run ends** the app keeps that run's steps as history, removes the test password from Claude Code's saved session, compares the result with the previous run of the same test and, for smoke tests, measures text contrast on the page.

2.5 Statuses

Status	Meaning
Pass	The step worked as expected.
Fail / Bug	A product defect. Every Fail step comes with a bug report (title, severity, priority, repro steps, expected, actual). Smoke tests call it *Bug*.
Blocked	Could not be checked: a captcha, a page outside the allowed domains, the environment down, missing data.
Not run	Planned but not reached.
Pending review	Claude's status is there, but you have not reviewed the step yet; it is not final.

Every step has two statuses: **Claude's status**, as Claude reported it, and the **final status**, which is set only by your review. Reports use the final status and show Claude's next to it.

3 Installing

Install two prerequisites once, then run the Setup program.

3.1 Requirements

What	Details
Windows	Windows 10 or 11, 64-bit. About 600 MB of disk space for the app.
Node.js 18 or later	Runs Playwright MCP (the browser tools). Install the LTS version from nodejs.org/en/download.
Claude Code	Signed in with a Claude **Pro or Max** subscription. In PowerShell: `irm https://claude.ai/install.ps1 | iex`, then open a new terminal and run `claude` once to sign in. Details: code.claude.com/docs/en/setup.
Internet	Claude Code needs to reach Anthropic. The application you test can be local (for example `localhost`) or online.

3.2 Install with Setup.exe

1. Download **WebTestingAssistant-Setup-1.1.3.exe** from the project's GitHub *Releases* page (or get it from whoever built it). Builds of branches and of `pre-release` are marked *Pre-release* there; see chapter 20 for the version numbers.
2. Double-click it. If Windows SmartScreen warns you, choose **More info** → **Run anyway**: the installer is not code-signed.
3. Click through the wizard. It installs for your user without administrator rights (an all-users install is offered). Tick *Create a desktop shortcut* if you want one.
4. Click **Finish** to start the app, or open **Web Testing Assistant** from the Start menu later.

3.3 First start: the setup check

On start the app checks that Node.js is installed and that Claude Code is installed and signed in. If something is missing, the **Check setup** window opens once and says exactly what to do; after fixing it, click **Check again**. You can open it any time from **Help** → **Check setup**. Until both are ready, Run batch stays disabled and its tooltip says why.

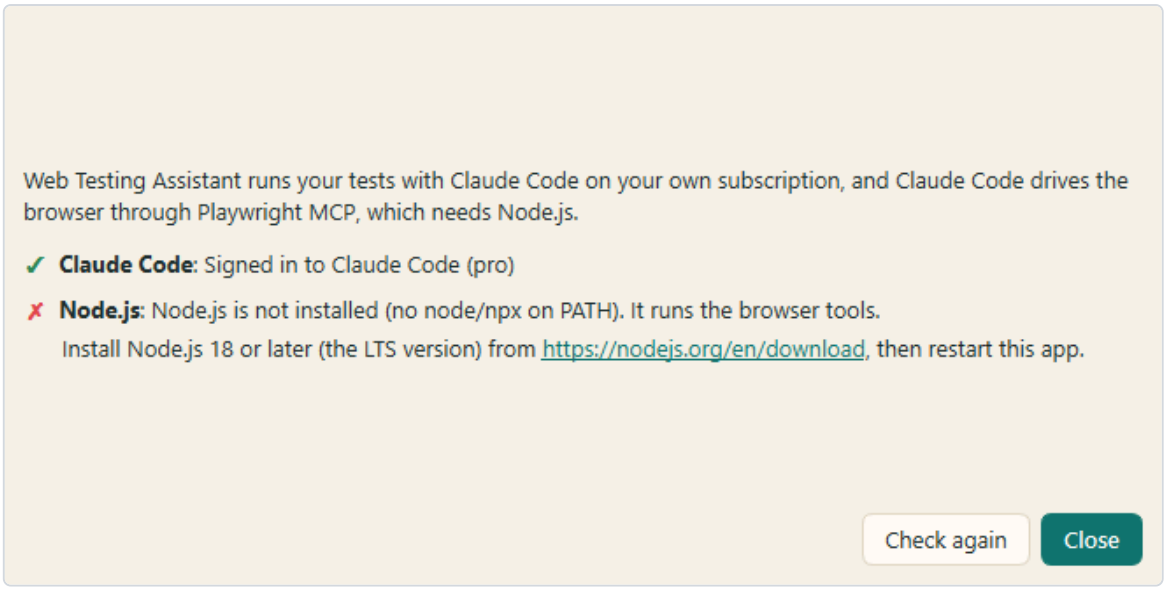


Figure 3. The setup check on a computer where Node.js is missing.

The top strip shows a green dot and *Claude Code signed in* when everything is ready, and the Launchpad's Test Case Design card shows *Ready*.

3.4 Updating and uninstalling

- **Update:** run a newer Setup.exe over the installed version. It closes a running copy first and keeps your data.
- **Uninstall:** *Settings → Apps → Installed apps → Web Testing Assistant → Uninstall*. Your sessions, runs, settings and backups stay in `%APPDATA%\Web Testing Assistant` until you delete that folder yourself.

> **Running from source instead**
> Developers can run the app from the repository with Python 3.12: see chapter 20.

4 Quick start

From an empty app to a reviewed test in about ten minutes.

1. **Open Regression Testing** on the Launchpad.
2. **Create a session. New session** on the toolbar (`Ctrl` + `N`). Enter a name, the target URL (for example your test environment's login page) and a test account. The target's host is always allowed; add other domains the test may visit. Click **Save**: the browser opens the target URL.
3. **Add a scenario.** On the **Scenarios** tab click **Add scenario…**. Write the steps one per line, and the expected result the test must reach. Example:

```
Steps:            Sign in with the test account
                  Click Profile in the header
                  Replace the display name with {{random_name}}
                  Click Save
Expected result:  The confirmation reads 'Profile saved. Display name: {{random_name}}'
```

4. **Run it.** Click **Run batch** (`F5`). The app switches to **Live run**; an amber banner over the browser shows that Claude is in control. Don't click in the page until the run ends.
5. **Review.** When Windows notifies you that the run finished, go back to **Scenarios**. Select each step and press `A` to approve Claude's status, or choose another final status and press `O` to override it.
6. **Export. Export** on the toolbar (`Ctrl` + `E`) writes an Excel, Word, PDF or Markdown report.

> **Starting from backlogs instead?**
> Choose **Test Case Design** on the Launchpad, upload your Excel workbook and let Claude write the test cases first (chapter 6). Hand them over as scenarios, then run them as above.

5 The Launchpad and the main window

Pick a mode, then work in a window made for it.

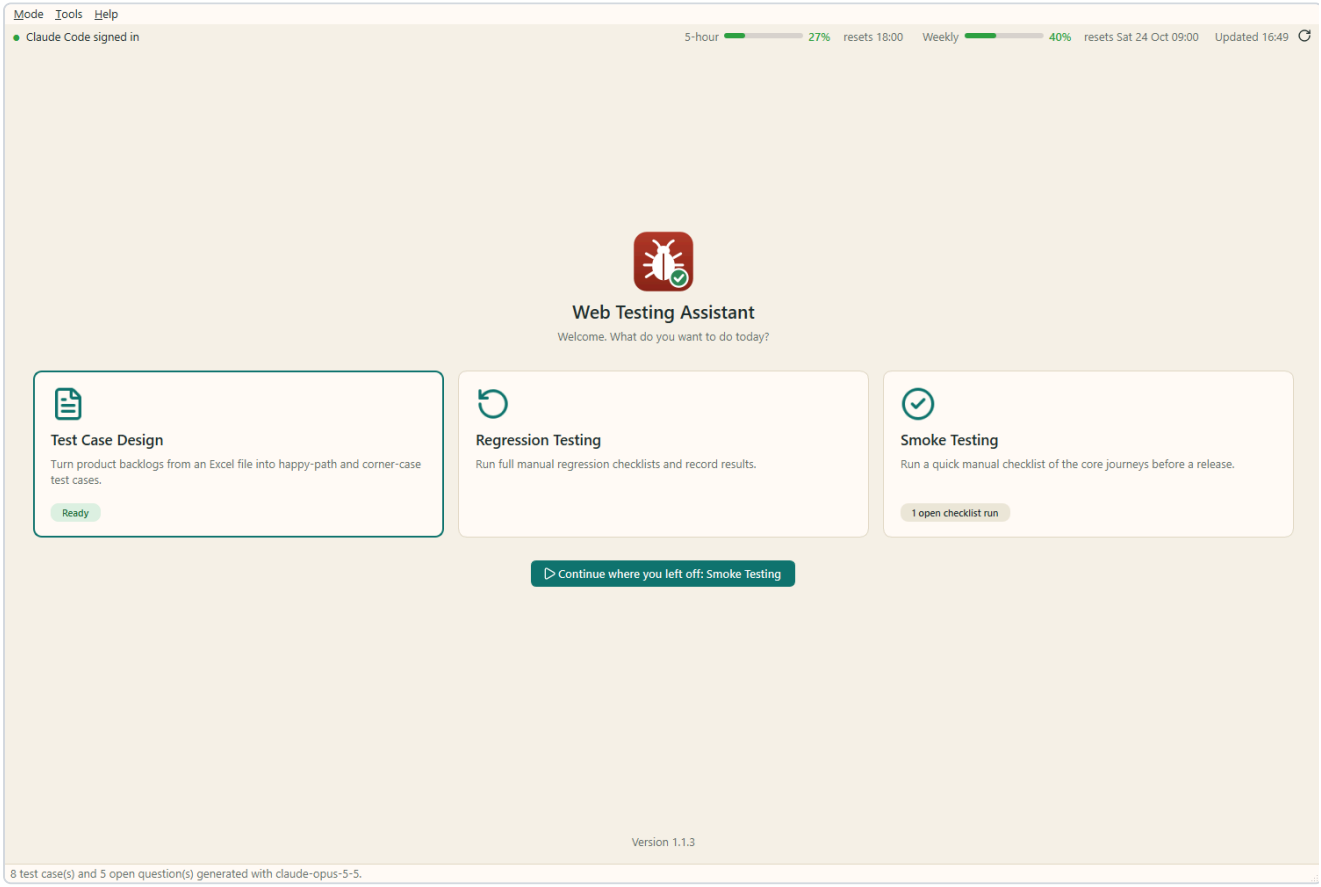


Figure 4. The Launchpad. Badges show whether Test Case Design is ready and how many checklist runs are open; *Continue where you left off* returns to the last mode.

5.1 The Launchpad and modes

- **Test Case Design** opens the workbook step, then the design workspace (chapter 6).
- **Regression Testing** shows the browser and the **Scenarios** tab: your normal scenarios.
- **Smoke Testing** shows the browser and the **Smoke test** tab: pages to smoke-test and their checks.

Each testing mode shows only its own review tab, next to **Live run**, **Runs** and **Checklists**. Switch modes with the mode button at the top or the **Mode** menu (`Alt` + `M`); **Back to Launchpad** is `Ctrl` + `L`.

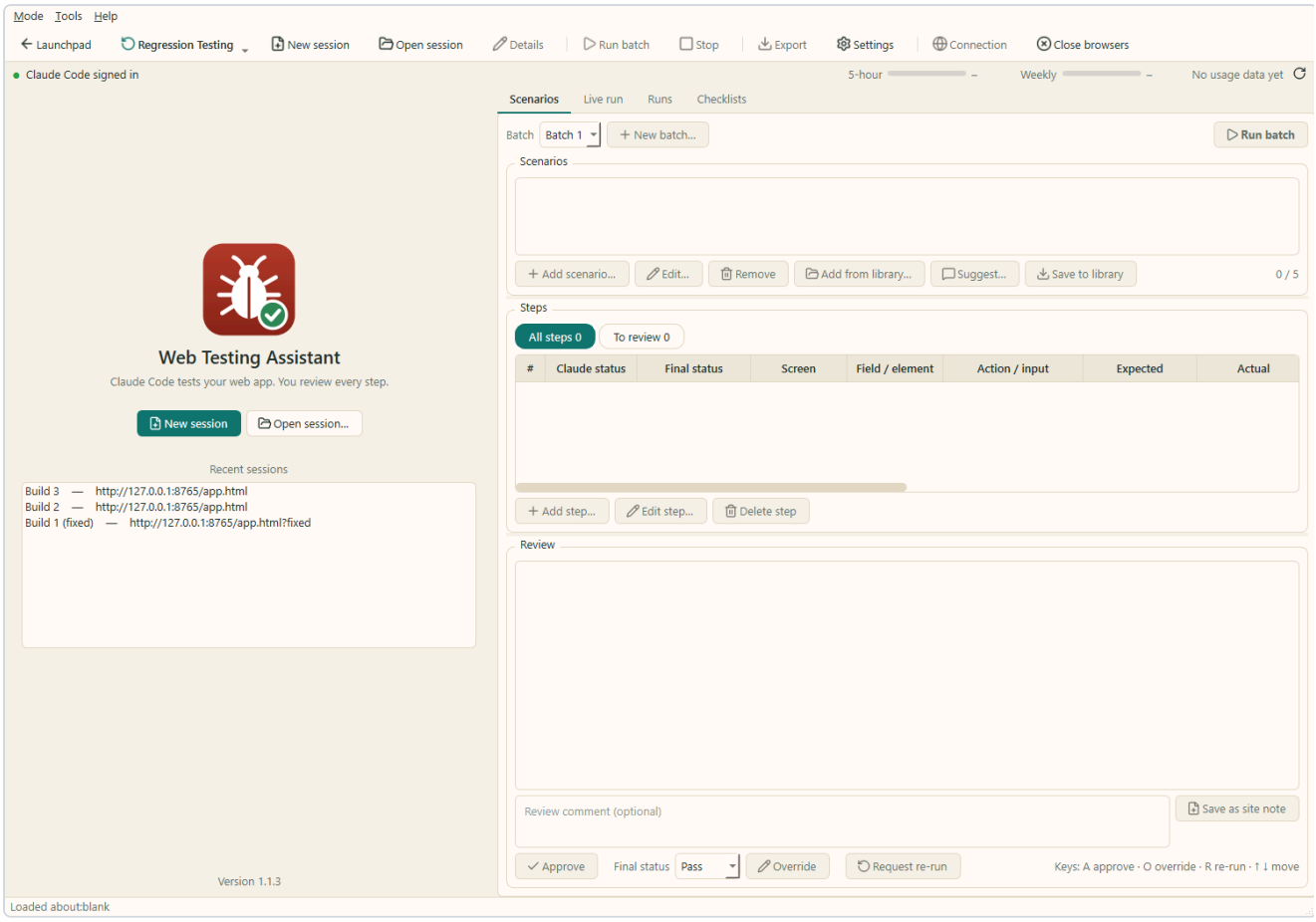


Figure 5. Regression Testing before a session is open: recent sessions are one click away, and the last session reopens with *Continue where you left off*.

5.2 Parts of the window

Part	What it does
Mode bar	**Launchpad** and the current mode, with a list of the others.
Toolbar	Only in Regression and Smoke: the session and browser actions (below).
Top strip	A green or red dot for Claude Code and Node.js, and the 5-hour and weekly plan usage with their reset times and a refresh button (section 5.5).
Browser	The application under test, at the session's viewport size. You can browse it yourself between runs.
Scenarios / Smoke test	Batches of tests, their steps and the review panel (one of the two, by mode).
Live run	The timeline of the run in progress.
Runs	Every past run with its trace, findings, comparison and the site notes.
Checklists	Manual checklist runs of designed test cases for this mode (chapter 11).

5.3 Resizing, rearranging and expanding

In Regression and Smoke the browser and the work panel share the tab. Drag the divider between them to give either more room; neither can be dragged away completely. The buttons at the top right of the work panel rearrange them:

Button	What it does
Swap	Puts the browser on the other side (or at the bottom, when stacked).
Stack / Side by side	Places the browser and the panel one above the other, or next to each other.
Reset	Browser left, panel right, half each.
Expand	Lets the panel fill the whole tab. The browser has its own Expand button at the end of its address bar. Click it again to bring both sides back at the sizes they had.

The layout is not saved: the app always opens with both sides, side by side, half each.

5.4 Menus and toolbar

Menu	Items
Mode	Test Case Design, Regression Testing, Smoke Testing, Back to Launchpad, Quit (`Ctrl` + `Q`).
Tools	Search everything, Trends, Clean up and back up, Settings.
Help	Check setup, About Web Testing Assistant.

Session and browser actions are only used when testing, so they are on the toolbar of Regression and Smoke, not in the menus:

Toolbar button	What it does
New session · Open session	Create a session or open an earlier one (`Ctrl` + `N`, `Ctrl` + `O`).
Details	Edit the open session: URL, domains, accounts, upload folder.
Run batch · Stop	Run the batch of the mode's tab (`F5`); stop at once (`Shift` + `F5`).
Export	Export the session's report (`Ctrl` + `E`).
Settings	App settings (`Ctrl` + `,`); also in Tools.
Connection	The browser's debugging endpoint and a command to connect Claude Code by hand.
Close browsers	Stop any run and close every process the app or Playwright started.

5.5 Plan usage and notifications

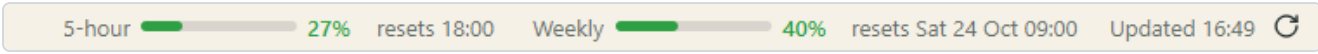


Figure 6. The usage gauges: percentage and reset time next to each bar, the time of the last reading, and the refresh button.

The gauges show what Claude Code reports. They update during every run and generation; the **refresh** button asks for a reading now with one tiny Claude Code call (Haiku, no tools), which uses a very small part of your plan. After a window resets, its old reading shows as – *needs refresh*. On a narrow window the reset times move to the tooltip.

Windows notifications come from **Web Testing Assistant** with its logo when a batch finishes or pauses, when Claude needs your confirmation, and when a test case generation finishes. They stay in the Windows notification center; the taskbar button also flashes until you look.

The app follows the Windows light or dark theme by default (you can fix it in Settings). The light theme is cream white with a teal accent, the dark theme uses brick red, and Fail/Bug statuses keep their own red in both.

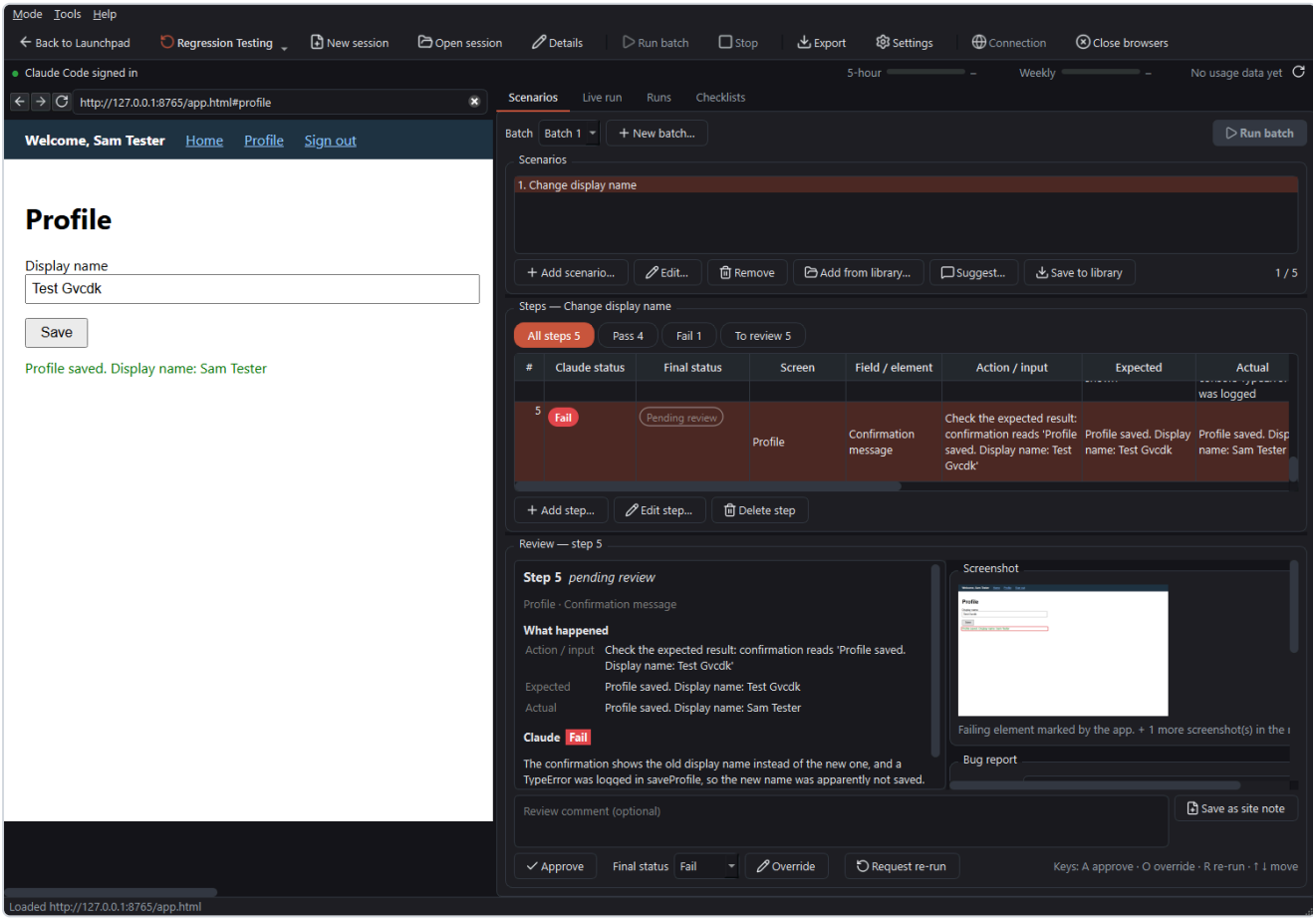


Figure 7. A review in the dark theme.

6 Test Case Design

From product backlogs in Excel to reviewed test cases, one backlog at a time.

Claude reads a backlog's text and its screenshots and writes **happy-path** and **corner-case** test cases, plus the questions it could not answer from the material. It runs through Claude Code on your subscription, with no tools and no browser; only the backlog being generated (text, pictures and your notes) is sent.

6.1 Workbooks, mapping and product context

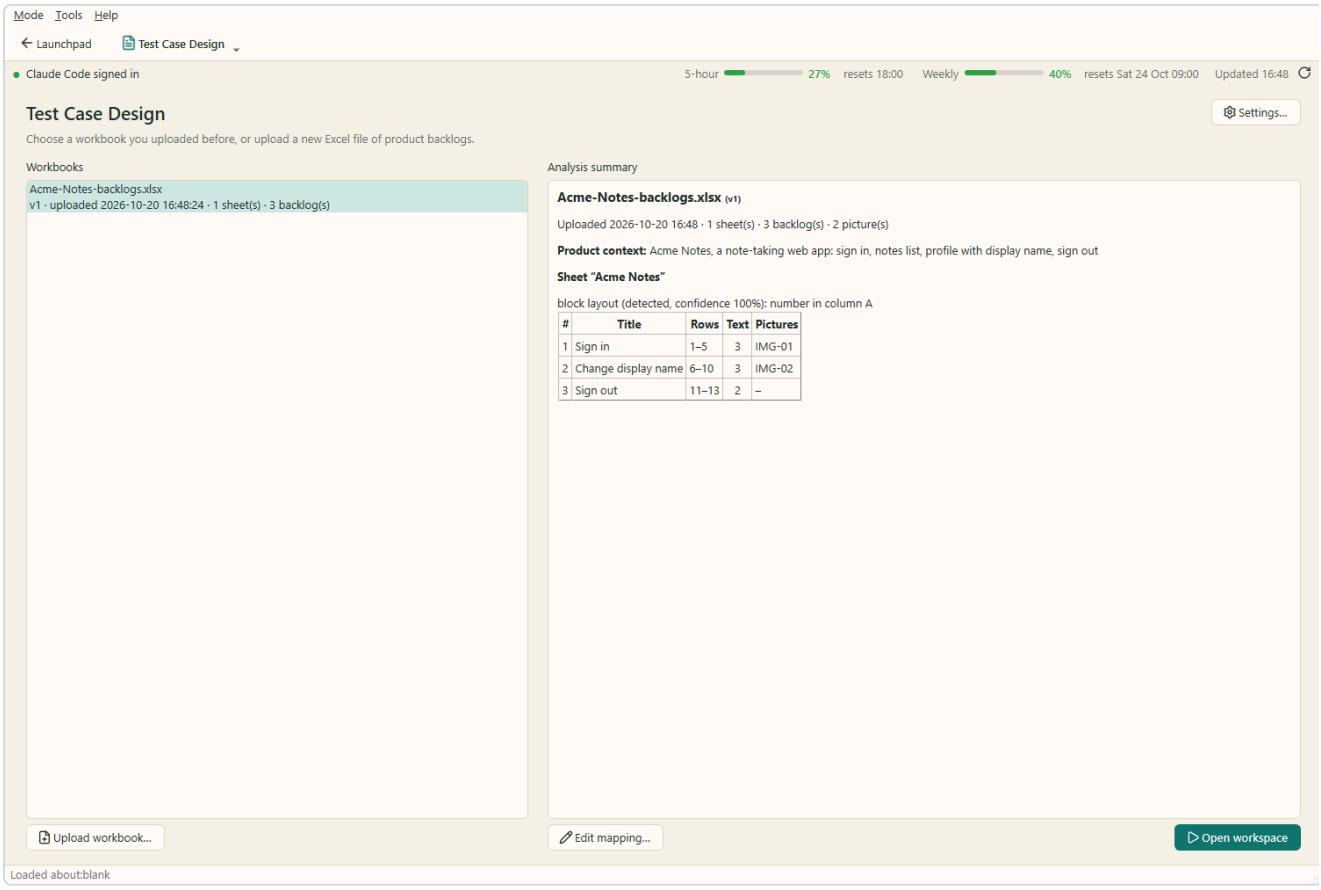


Figure 8. The workbook step: uploaded workbooks and the analysis summary with the product context, the detected layout and every backlog with its pictures.

1. **Upload workbook…** reads an `.xlsx` (or `.xlsm`) at once. The summary shows each sheet's layout (*block*: a number row with description rows below; *table*: a header row and one backlog per row), how confident the detection is, every backlog with its rows and pictures (IMG-01 …), and warnings.
2. **Edit mapping…** sets the **product context** of the workbook and, if a sheet was read wrongly, its columns and rows. *Use automatic detection* undoes a hand-set mapping.
3. A newer version of the same file becomes v2, v3 …: the summary lists the backlogs added, removed or changed, the product context and hand-set mappings carry over, and test cases, history and notes follow each backlog.
4. **Open workspace** starts work on the chosen workbook.

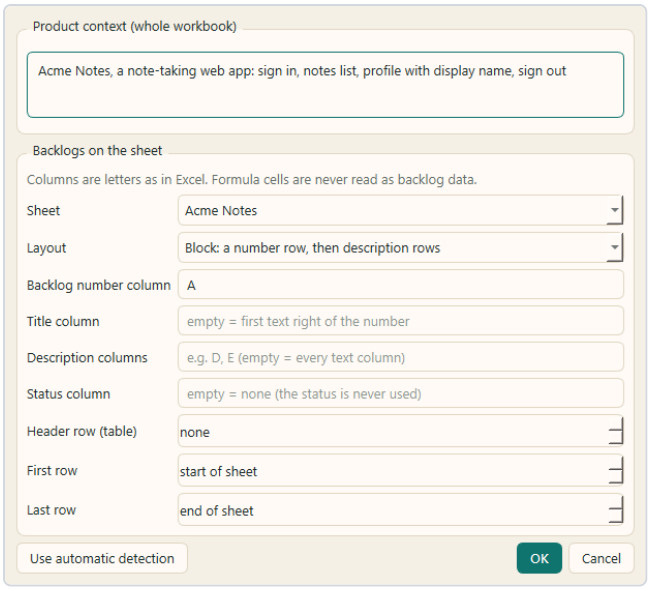


Figure 9. The product context belongs to the workbook; the mapping to a sheet.

The **product context** tells Claude what the application under test is, for example "*Acme Notes, a note-taking web app: sign in, notes list, profile, sign out*". It fills "the product backlogs of ..." in the prompt for every backlog of the workbook, so each workbook describes its own product. A workbook without one uses the value saved by an earlier version of the app.

Formula cells and the status column are never sent.

6.2 Pick a backlog and check what is sent

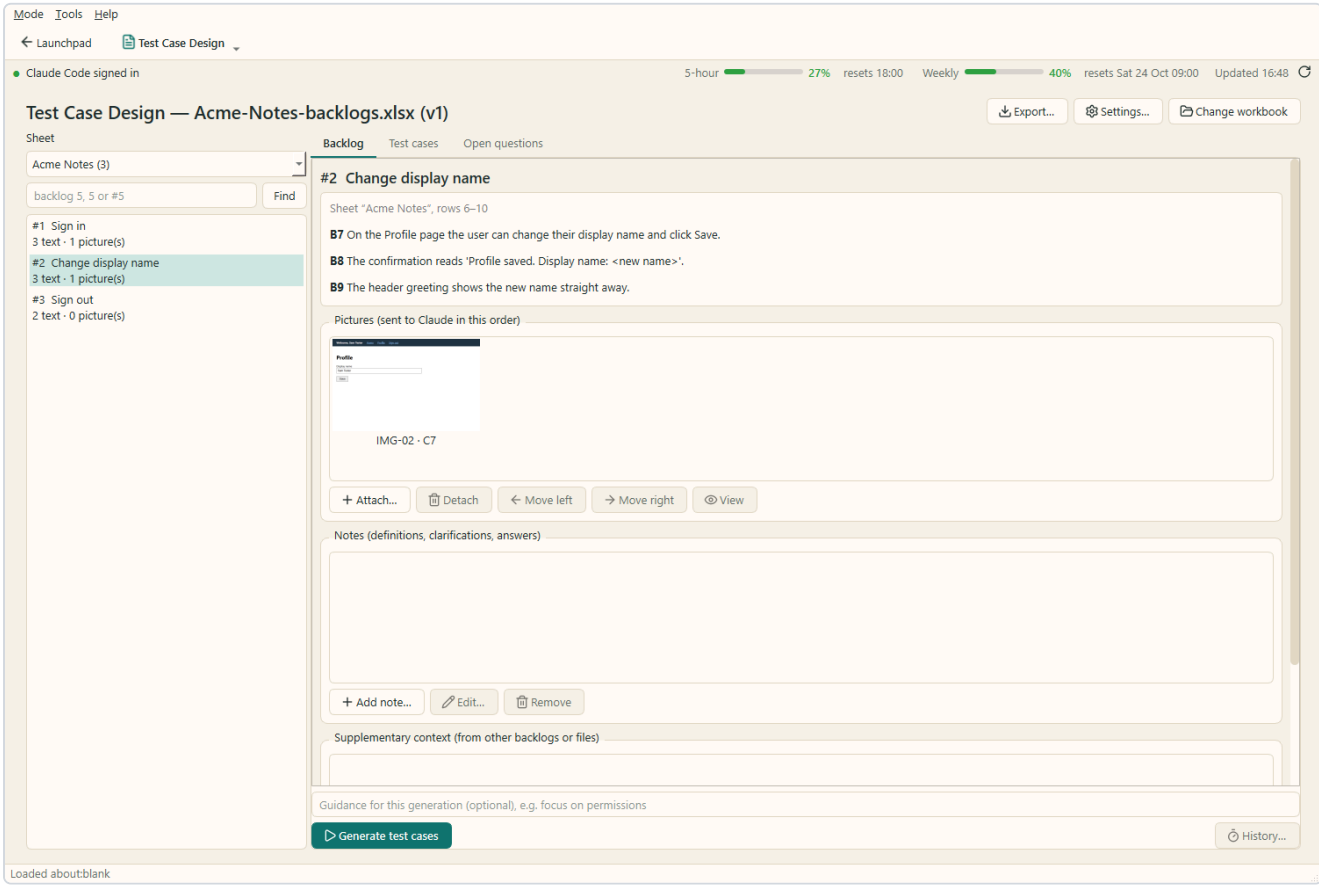


Figure 10. The Backlog tab: the description with its cells, the pictures in the order they are sent, notes and supplementary context.

- Choose the sheet and type the number: `backlog 5`, `5`, `#5` and codes like `BL-5` work. Matches are exact, so `1` never finds `10`.
- **Pictures:** attach any picture of the workbook, detach, reorder, view full size.

- **Notes:** definitions, clarifications and answers to earlier questions, sent with every generation of the backlog.
- **Supplementary context:** another backlog's text, or text you paste, when behaviour is described elsewhere.

6.3 Generate, review and edit

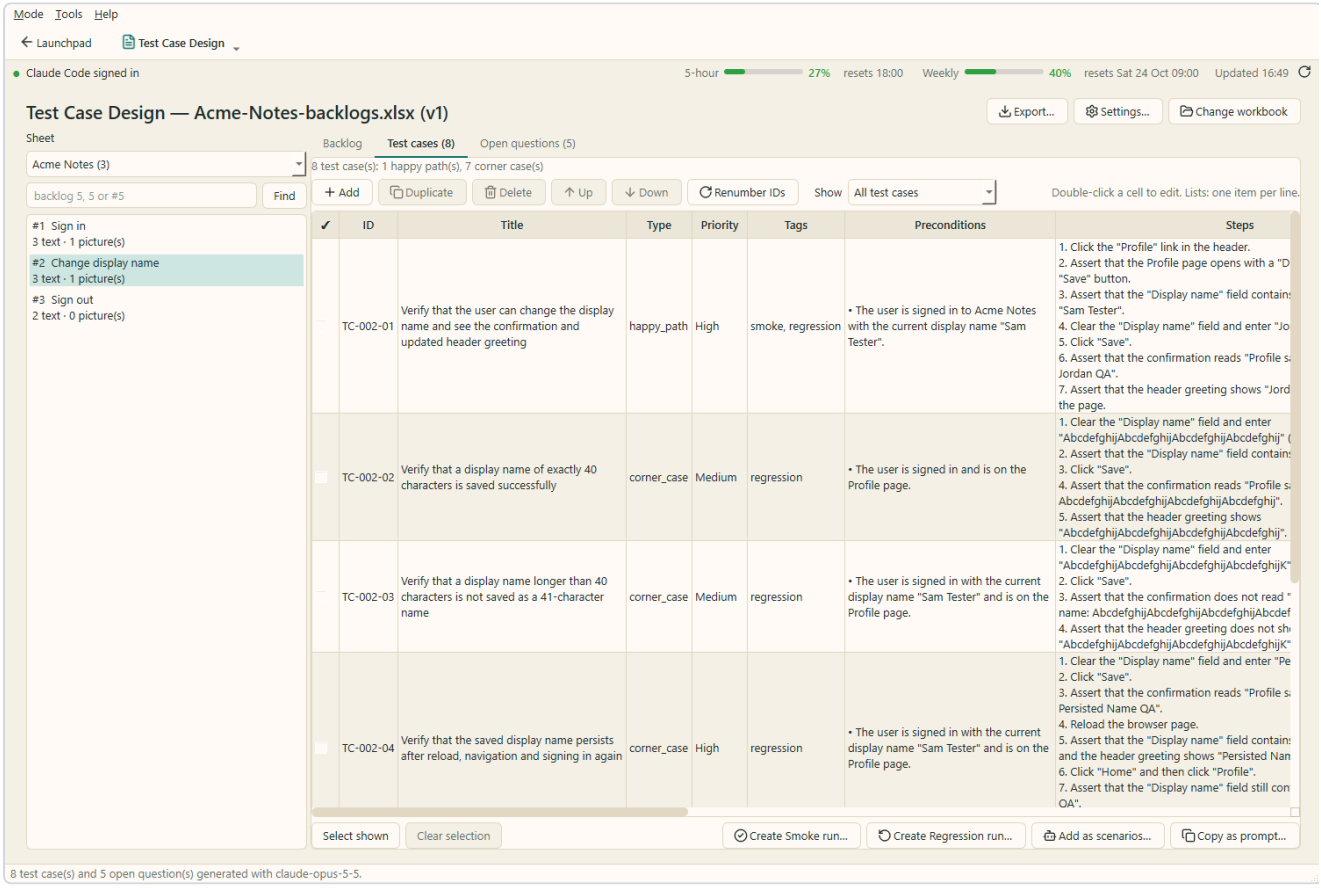


Figure 11. Eight test cases for "Change display name", generated with Claude Opus; five open questions are on the next tab.

- **Generate test cases** takes a minute or two. The answer is checked against the schema and the rules (IDs, happy paths first, an "Assert that" check in every case, tags); a failed check is sent back once for a fix, and a cut-off answer is continued. A notification tells you when it is done.
- **Edit** any cell; Type, Priority and Basis are lists, preconditions and steps are one per line. Edited cases show ✎. Add, Duplicate, Delete, Up, Down and Renumber IDs change the list. Warnings point out what to fix but never block.
- **Regenerate** with optional guidance ("more permission cases"). Lock the cases to keep, and Claude keeps them and adds what is missing. **History...** lists every generation; **Restore this run** brings one back.
- **Open questions:** type an answer and **Add answer to notes**, so the next generation uses it.

6.4 Export and hand over

Action	Result
Export...	This backlog or the whole workbook, optionally one tag only, as JSON, CSV (UTF-8 with BOM) or Excel ("Test Cases" and "Open Questions" sheets). Columns follow the output preset.
Create Smoke run... / Create Regression run...	A manual checklist run of the ticked cases, new or added to an open run (chapter 11).

Add as scenarios…	The cases go into the scenario library; in Regression Testing use **Add from library…** so Claude Code runs them in the browser.
Copy as prompt…	A ready-made prompt with the cases on the clipboard, to paste into Claude for a printable checklist.

6.5 Test Case Design settings

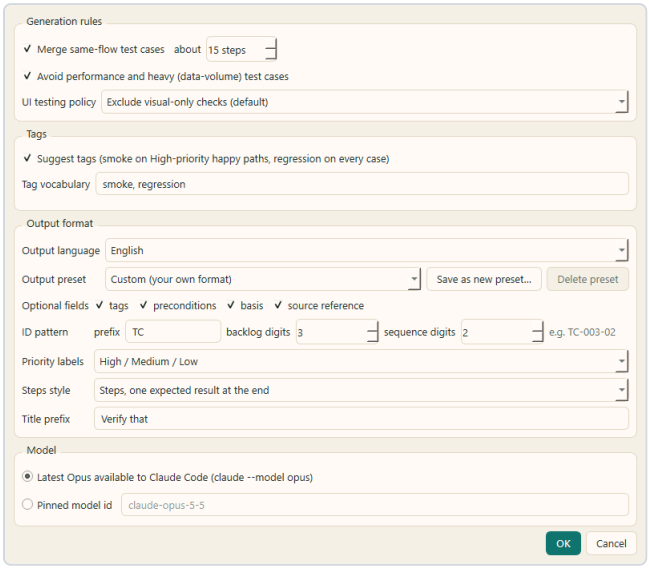


Figure 12. Settings… on the workbook step or in the workspace, here with the Custom preset.

Setting	Default
Merge same-flow cases	On, about 15 steps
Avoid heavy cases	On: no load, speed or large-data cases
UI testing policy	Exclude visual-only checks
Suggest tags	On: `smoke`, `regression`
Output language	English (more languages later)
Output preset	Default (fixed). **Custom** makes the ID pattern, priority labels, steps style and title prefix editable; *Save as new preset…* keeps more.
Model	Latest Opus, or a pinned model id

7 Sessions

A session is one testing assignment: one application, its accounts and its rules. Sessions are used by Regression and Smoke Testing; create one with **New session** on the toolbar and edit it with **Details**.

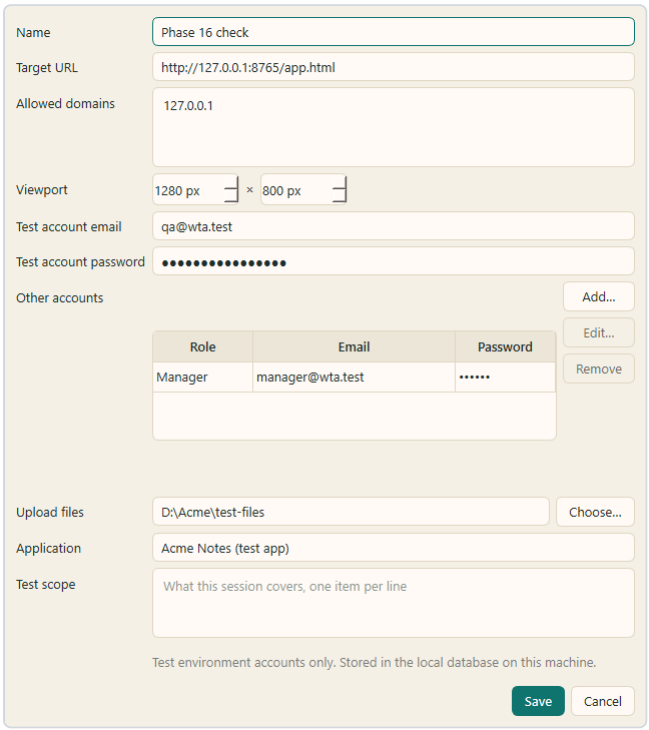


Figure 13. Session details with a second account (role) and an upload folder.

Field	Purpose
Name	For you; for example the build or sprint.
Target URL	Where every run starts.
Allowed domains	The only hosts the browser may open during a run; the target's host is always included.
Viewport	The page size in CSS pixels (default 1280 × 800).
Test account	Email and password of a **test-environment** account.
Other accounts	More accounts with a role name, for example *Manager* or *Team B*.
Upload files	A folder of files Claude may upload.
Application, Test scope	Shown in the report's summary.

7.1 Test accounts and roles

Each scenario or smoke page can **run as** any account of its session (the *Run as* list appears when there is more than one). The prompt names the role, and Claude signs in as that user. The browser keeps its sign-in between scenarios, so Claude checks first and signs in only when needed; when a scenario tests signing in or out itself, it signs out first and follows the steps exactly.

> **Test accounts only**
> Use accounts of a test environment. Account details stay in the local database on this computer. Every password of the session is removed from run files, Claude Code's saved sessions and reports; the account email may appear in reports.

7.2 File uploads

Without an upload folder, Claude cannot upload anything. With one, the app copies its files into each run folder (`fixtures/`; hidden files skipped; at most 25 MB per file and 200 MB per run) and allows the upload tool for those files only. The prompt lists them, so Claude can pick the right one.

8 Scenarios and smoke tests

Write tests the way you already do: steps and an expected result.

8.1 Scenarios

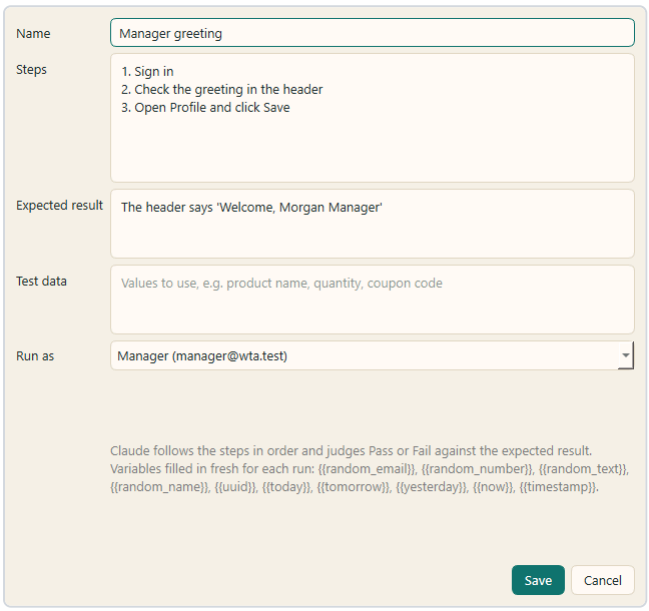


Figure 14. A scenario that runs as the Manager account.

- **Steps:** one per line. Numbering is added automatically. Claude follows them in order and records one step per line, keeping your wording.
- **Expected result:** what must be true at the end. Claude records the check as its own final step: *Pass* only if it is met exactly.
- **Test data:** values the steps need. You may use **variables** (below).
- **Run as:** the account to sign in with.
- Leave the steps empty and Claude plans them itself from the name and the expected result.

Test data variables

Variables are filled in fresh for each run, with the same value everywhere in that run, so data from earlier runs never clashes. The values used are listed at the top of the run's trace.

Variable	Example value
`{{random_email}}`	wta.kdlspq123@example.com
`{{random_name}}`	Test Gvcdk
`{{random_number}}` · `{{random_text}}`	482913 · hqzmwpxa
`{{uuid}}`	3f2b8c1e-…
`{{today}}` · `{{tomorrow}}` · `{{yesterday}}`	2026-10-19 · 2026-10-20 · 2026-10-18
`{{now}}` · `{{timestamp}}`	2026-10-19 09:05 · 20261019090512

Batches

A batch holds up to **5** scenarios. **Run batch** runs every scenario of the batch that has no results yet or whose steps you sent back for a re-run. A new batch can start once the current one is reviewed; the app asks for confirmation because each batch uses plan.

8.2 Smoke tests

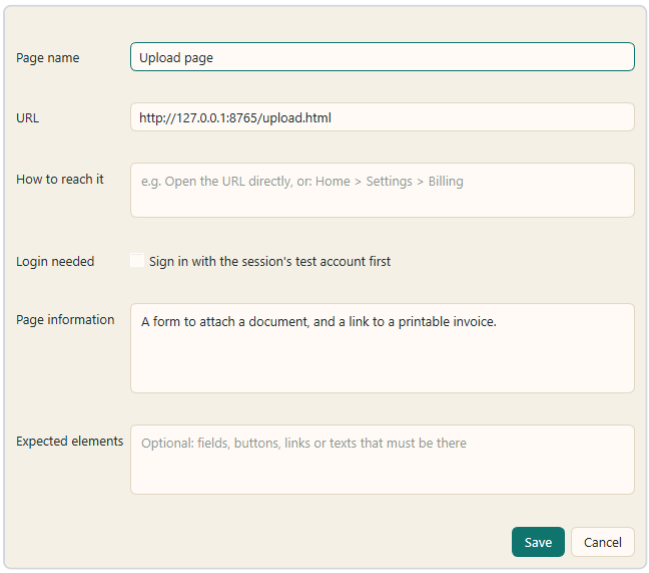


Figure 15. Describing one page for a smoke test.

A smoke test checks one page broadly. Describe the page: its URL, how to reach it, whether a login is needed, what is on it and which elements you expect. Claude first writes the whole checklist (usually 10 to 20 checks), then runs the checks in order: the page loads, every expected element is present and labelled, the main controls work, links point to the right places, there are no console errors.

With *Include accessibility checks* (on by default), Claude adds checks for form labels, button and link names, image alt text and the main heading, and the app measures text contrast on the page after the run (chapter 12).

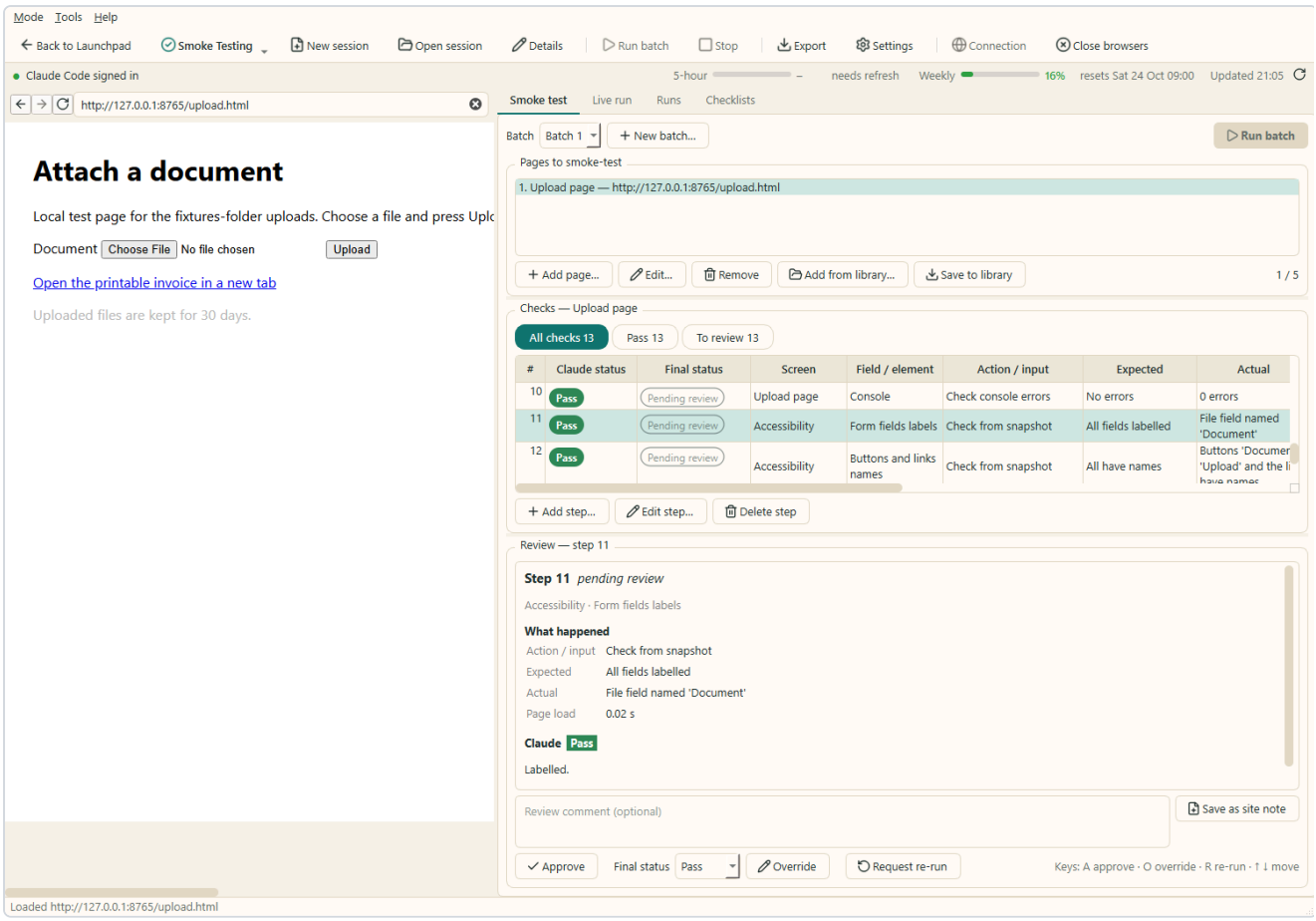


Figure 16. A smoke test's checks; the selected one is an accessibility check.

8.3 The library and importing test cases

Keep tests you will run again in the **library**: **Save to library** stores the selected scenario or smoke page, and **Add from library...** adds saved ones to the current batch, in any session. Test cases handed over from Test Case

Design with *Add as scenarios…* are in the library too. A scenario added from the library stays linked to it, so a run on the next build is compared with the run on the previous build (chapter 12).

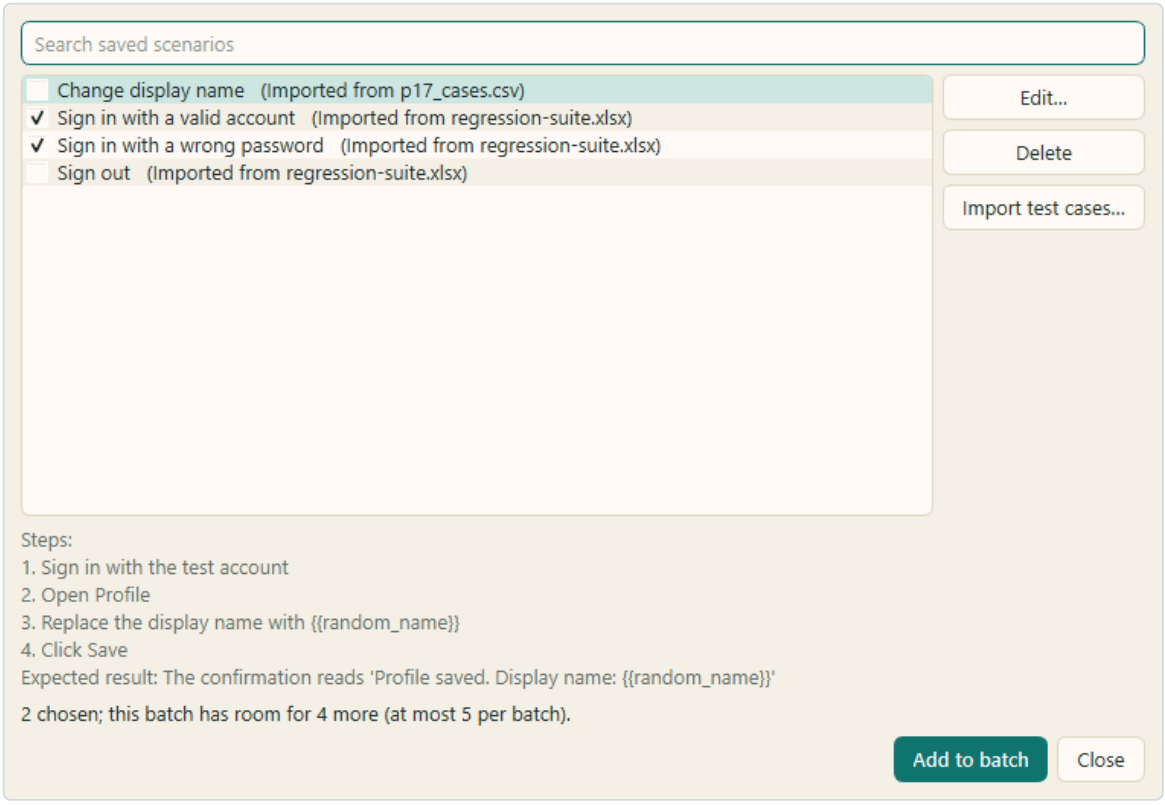


Figure 17. The library: tick the tests to add; the batch's free places are counted.

Import test cases… (in the library) reads an Excel (.xlsx) or CSV file. Two layouts are recognised by their column names:

Layout	Columns
One row per test case	*Name* or *Title*; *Steps* (one per line); *Expected result*; *Test data*.
Azure DevOps export	A *Title* row followed by one row per step with *Step Action* and *Step Expected*. HTML in the cells is removed; the last step's expectation becomes the expected result.

A preview lists the test cases found before anything is added.

8.4 Let Claude suggest scenarios

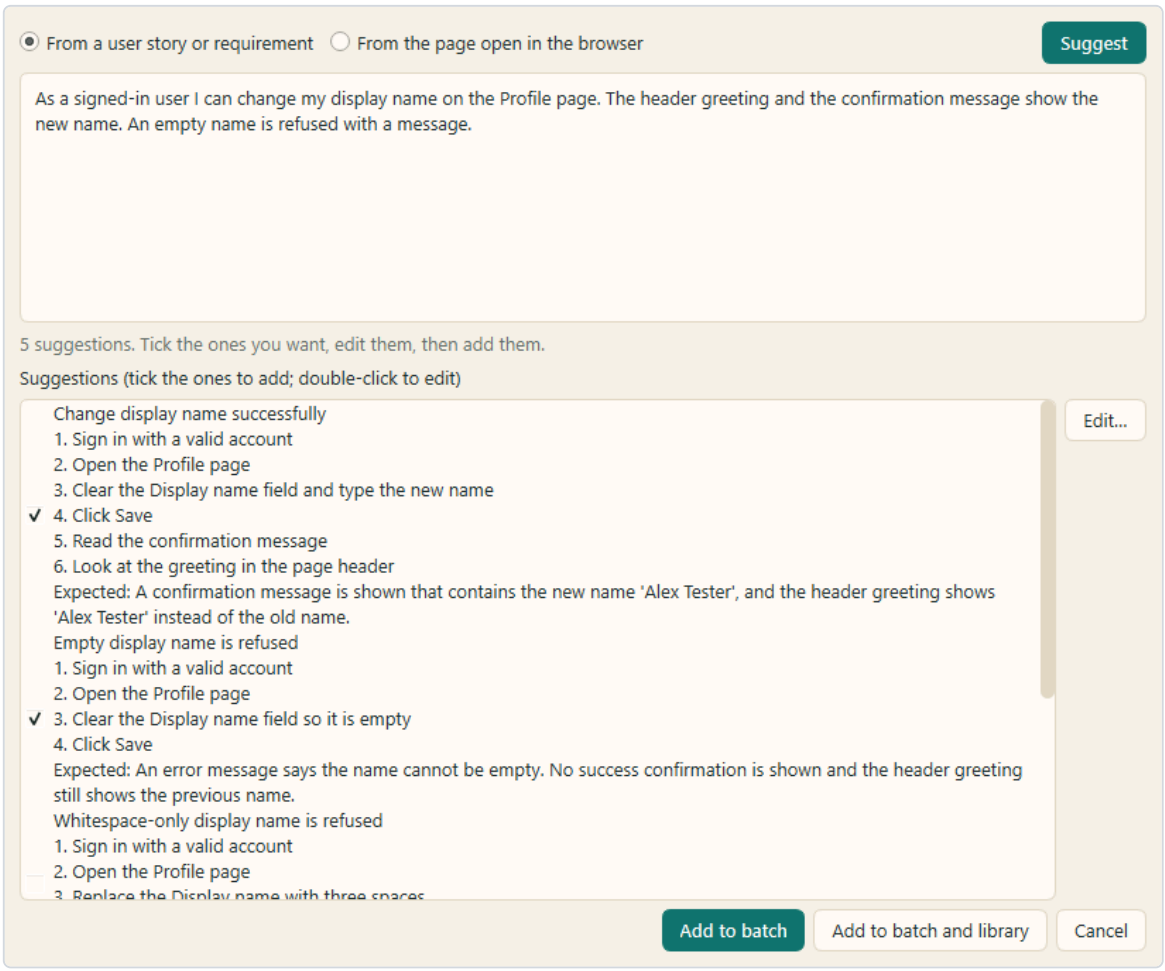


Figure 18. Five suggestions from a short user story; tick, edit and add the ones you want.

Suggest... on the Scenarios tab asks Claude for 3 to 5 scenarios from a pasted user story or from the text of the page open in the browser. This short request gives Claude no tools at all and is not saved in Claude Code's history; the text is treated as data. Double-click a suggestion to edit it, then add the ticked ones to the batch, optionally to the library as well.

9 Running tests

Press Run batch and watch Claude work.

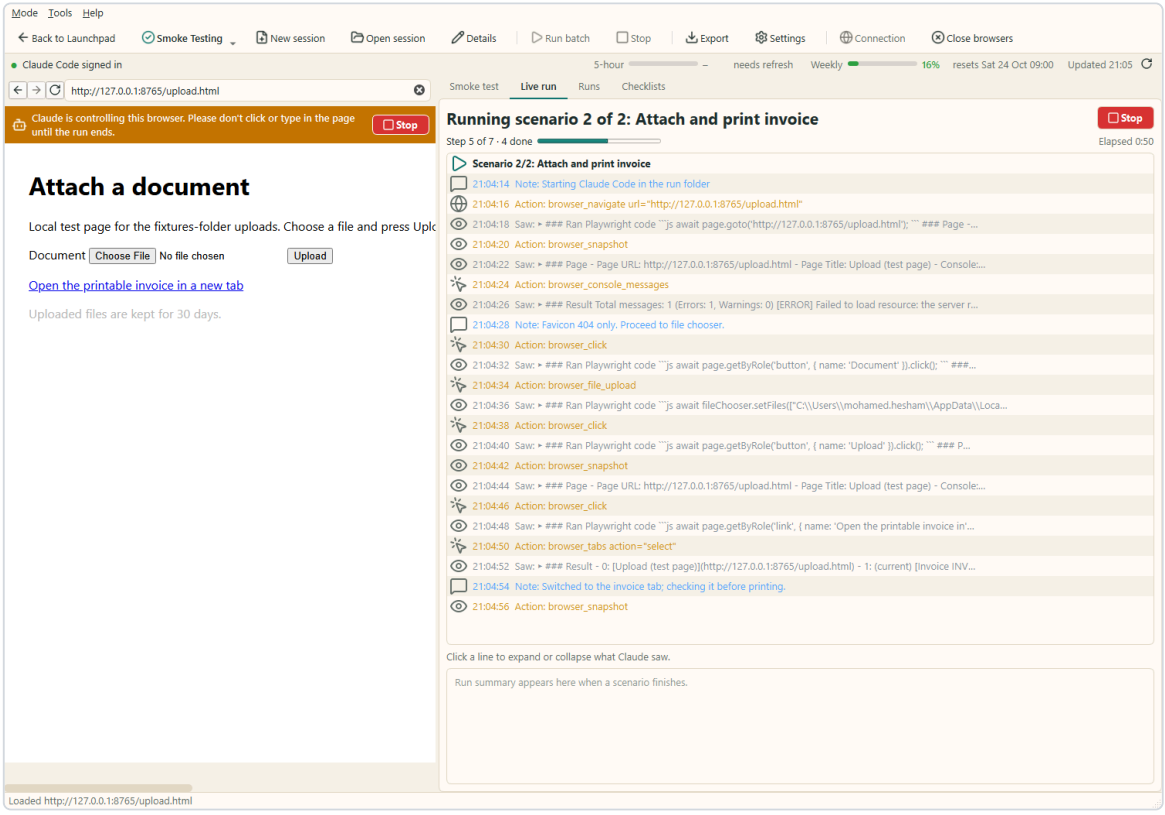


Figure 19. A run in progress: the banner over the browser, the timeline, progress and the elapsed time.

9.1 The live run

- **Timeline:** one line per note, action and result, with an icon per kind of action (navigate, click, type, screenshot, file). Long output from the page is shown on one line; click it to expand.
- **Progress:** "Step 5 of 7 · 4 done", from the steps Claude has written so far, and the elapsed time.
- **Banner:** while Claude is in control, an amber banner over the browser asks you not to click in the page, and has its own Stop button.
- **Summary:** when a scenario ends: turns, time, tokens, the model used, plan usage, notices, what the app saw on the page and what changed since the previous run.
- **Notifications:** Windows notifies you, as Web Testing Assistant, when a batch finishes, pauses, or Claude asks for confirmation; the notifications stay in the notification center and the taskbar button flashes.

9.2 Staying in control

Control	What it does
Stop	Ends Claude Code and everything it started at once; the steps reached so far are kept.
Close all browsers	Stops any run and closes every process the app or Playwright started, tracked by process ID. Your own browsers are never touched.
Allowed domains	During a run the browser refuses pages outside the session's domains; Claude marks such a step Blocked.
Pause before risky actions	Optional (Settings, off by default). Before deleting, sending, paying or another irreversible action, Claude stops and the app asks you to allow or decline; the run then continues.
Pause at the usage limit	Before each scenario after the first, the batch pauses and asks you once your 5-hour usage has reached the limit (80% by default).

9.3 New tabs, pop-ups and printing

When the page opens a new tab or pop-up (a document, an invoice), it appears in its own window; Claude switches to it with the tabs tool, checks it and closes it. When the page prints during a run, the app saves the printout as `print-1.pdf` in the run folder, and Claude reads it to check it.

10 Reviewing results

Nothing counts until you have looked at it.

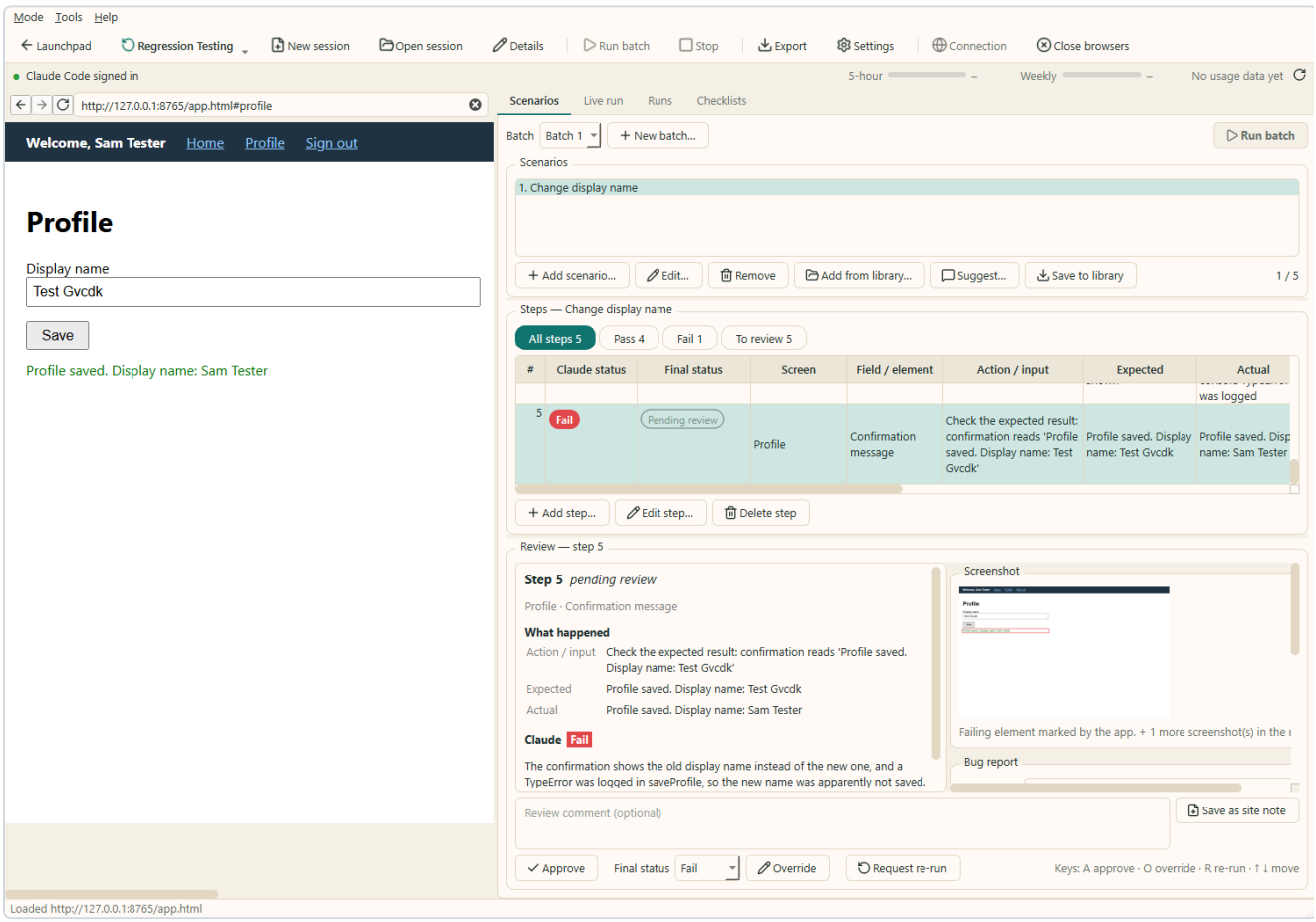


Figure 20. Reviewing a failed step: the browser still shows the page, the step card explains what happened, and the screenshot has the failing element marked.

10.1 The review panel

- **Filter chips** above the steps count them by status ("All steps 5 · Pass 4 · Fail 1 · To review 5"); click one to show only those steps.
- **Status badges** show Claude's status and the final status. Pending reviews and re-run requests are outlined.
- **Step card:** the screen and element, what was done, expected and actual, the page load time, Claude's status and reasoning, and the review history.
- **Screenshot:** click the thumbnail to see it full size. For Bug and Blocked steps the app adds its own copy with the failing element boxed in red, shown first.
- **Bug report:** every field is editable. Your version is what reports use; *Use Claude's version* brings back the original.

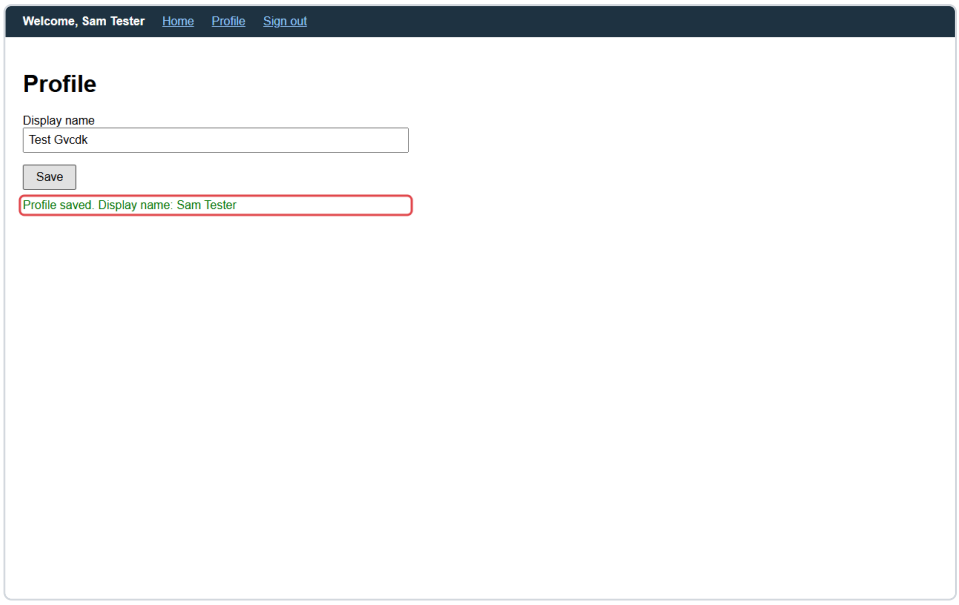


Figure 21. The app's marked screenshot: Claude named the wrong confirmation message, the app boxed it.

10.2 Deciding

Action	Key	Effect
Approve	A	Claude's status becomes the final status.
Override	O	Sets the final status chosen in the list; Claude's status is kept next to it.
Request re-run	R	Sends the step back; its scenario runs again on the next Run batch.
Next / previous step	↓ ↑	Move through the steps shown by the filter.

Keys work while the step table or a button has focus, never while you type. An optional comment is saved with each decision.

10.3 Site notes

Save as site note keeps your comment as a note for the site (the host name). Notes are added to the prompt of every following run on that site, for example "The coupon field needs a second click". They come from you, so Claude follows them; you manage them on the Runs tab.

11 Manual checklist runs

Designed test cases, run by you, with the results recorded.

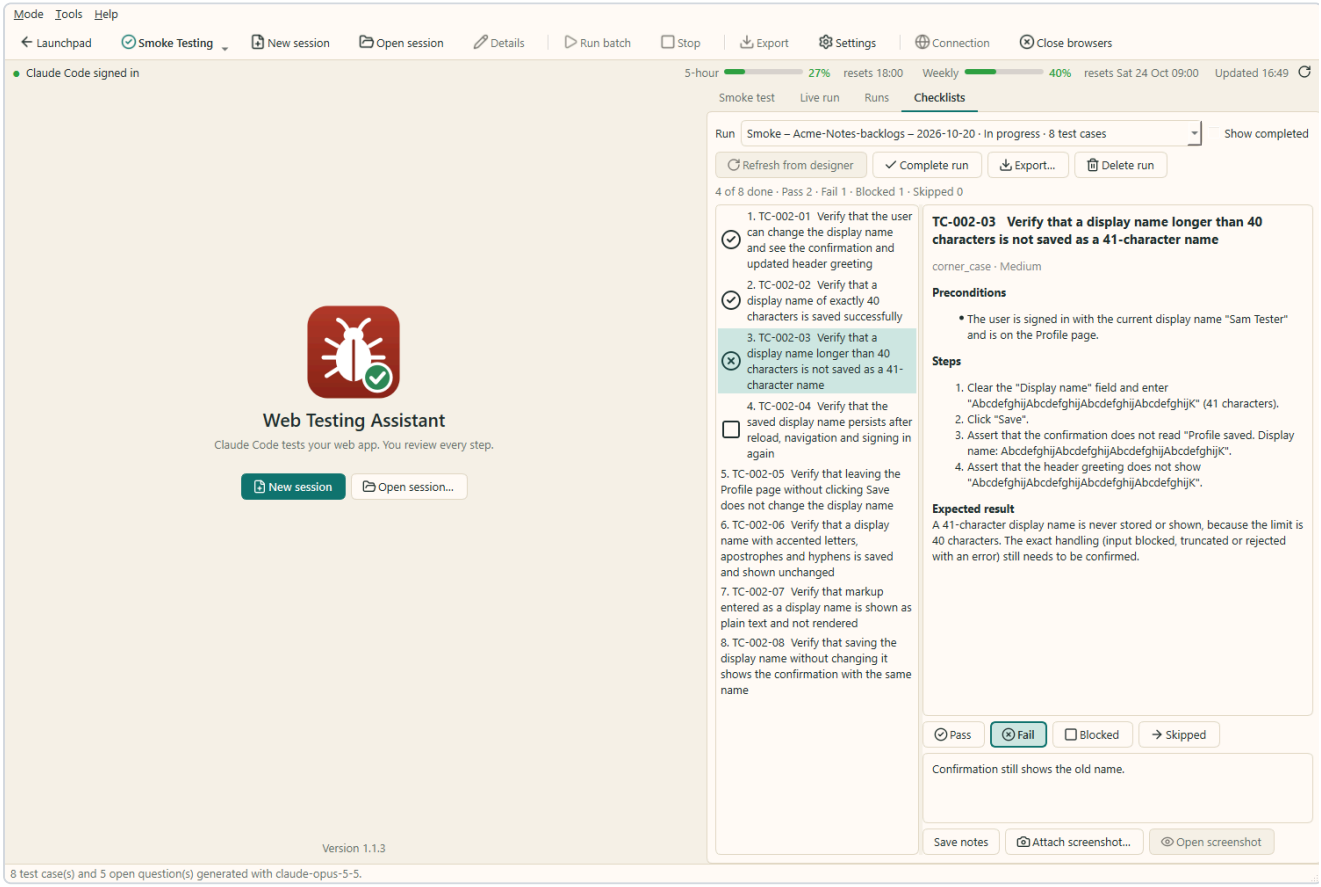


Figure 22. A Smoke checklist run from the designed test cases: 4 of 8 done, one failed with a note.

- Runs are created from Test Case Design (*Create Smoke run…* or *Create Regression run…*) and appear on the **Checklists** tab of that mode. The Launchpad cards count the open runs.
- For each test case you see the ID, title, preconditions, numbered steps and expected result. Record **Pass**, **Fail**, **Blocked** or **Skipped**; the next case is selected for you. Add notes, and **Attach screenshot…** for failures.
- The progress line counts the results. **Export…** writes the run to Excel or CSV. **Complete run** closes it; **Reopen run** opens it again.
- Each case is a snapshot, so later edits in the designer do not change the run. While a run has no results yet, **Refresh from designer** takes the current versions.

12 Runs, history and comparison

Every run is kept: what Claude did, what it reported and what the page did.

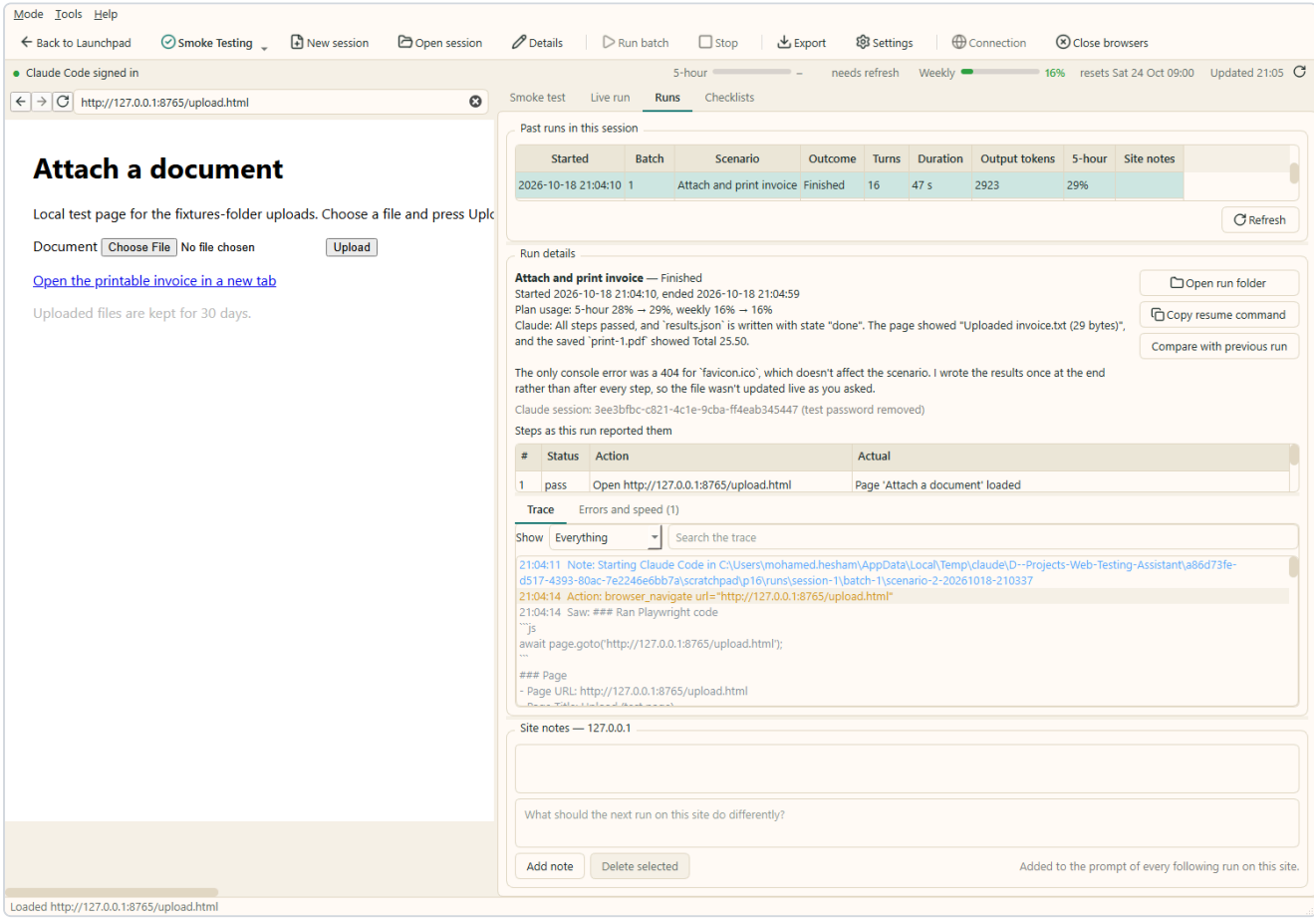


Figure 23. The Runs tab: past runs, the run's details and steps, its trace, and the site notes.

- **Run details:** outcome, start and end, plan usage, the site notes used, Claude's summary and the Claude session.
- **Steps as this run reported them:** each run keeps its own steps, so a re-run never erases history.
- **Trace:** every note, action and result, filterable and searchable.
- **Open run folder** and **Copy resume command**: the command reopens the run's Claude session in a terminal to ask about it (Claude gets no tools there; the test password has already been removed).

12.1 Errors and speed

What the app itself saw on the page during the whole run, including steps that passed: console errors and warnings, uncaught JavaScript errors, failed requests (HTTP 400 and above, or failed), page load times (slow from 3 seconds) and, for smoke tests, low text contrast. This uses no plan.

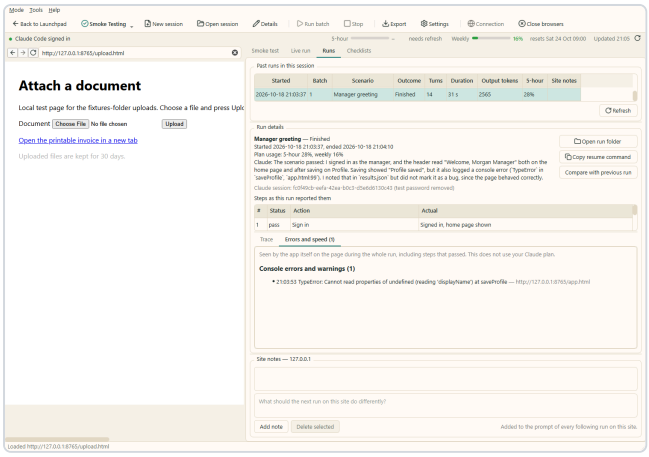


Figure 24. A console error on a step Claude marked Pass.

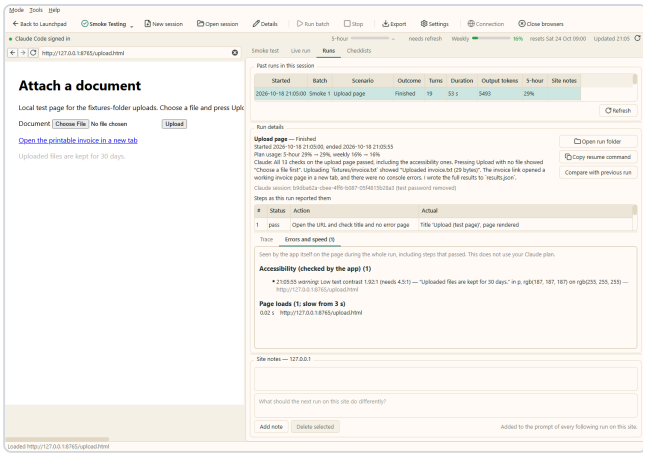


Figure 25. A smoke test: low contrast found by the app, and the page load time.

12.2 Compare with the previous run

The previous run of a test is the latest earlier run of the same scenario or, for scenarios added from the library, of the same library item in any session: one session per build compares build with build. Steps are matched by their words, then by position.



Figure 26. Build 2 against Build 1: one step that passed now fails.

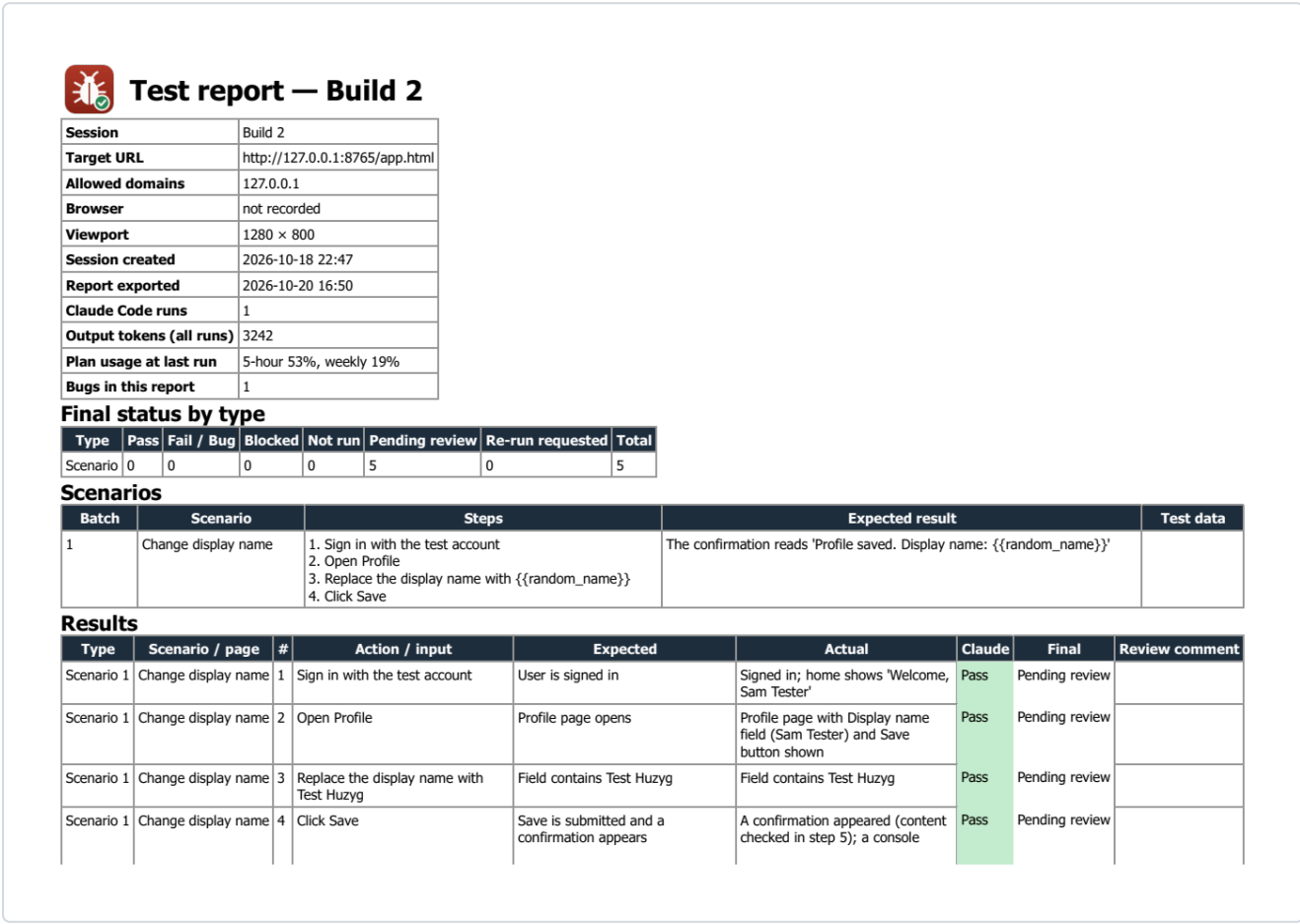
The live run summary shows the same comparison after each scenario, and the batch notification mentions regressions.

13 Reports

Hand results over without retyping them.

Export on the toolbar (`Ctrl` + `E`) exports the open session in the default format from Settings; choosing a file name with another extension picks that format.

Format	Contents
Excel	Summary, Results and Bugs sheets with screenshots. The Bugs sheet uses Azure DevOps / TFS column names (Work Item Type, Title, Repro Steps, Severity, Priority, Tags) for import.
Word, PDF	The same report as a document: session details, counts by status, scenarios with their steps, results, and every bug with its screenshots.
Markdown	A folder with `SUMMARY.md` (test information, scope, results per test case, overall result, bugs found, attachments), one `BUG-NNN-title.md` per bug, and the screenshots.



Test report — Build 2

Session	Build 2
Target URL	http://127.0.0.1:8765/app.html
Allowed domains	127.0.0.1
Browser	not recorded
Viewport	1280 × 800
Session created	2026-10-18 22:47
Report exported	2026-10-20 16:50
Claude Code runs	1
Output tokens (all runs)	3242
Plan usage at last run	5-hour 53%, weekly 19%
Bugs in this report	1

Final status by type

Type	Pass	Fail / Bug	Blocked	Not run	Pending review	Re-run requested	Total
Scenario	0	0	0	0	5	0	5

Scenarios

Batch	Scenario	Steps	Expected result	Test data
1	Change display name	1. Sign in with the test account 2. Open Profile 3. Replace the display name with {{random_name}} 4. Click Save	The confirmation reads 'Profile saved. Display name: {{random_name}}'	

Results

Type	Scenario / page	#	Action / input	Expected	Actual	Claude	Final	Review comment
Scenario 1	Change display name	1	Sign in with the test account	User is signed in	Signed in; home shows 'Welcome, Sam Tester'	Pass	Pending review	
Scenario 1	Change display name	2	Open Profile	Profile page opens	Profile page with Display name field (Sam Tester) and Save button shown	Pass	Pending review	
Scenario 1	Change display name	3	Replace the display name with Test Huzyg	Field contains Test Huzyg	Field contains Test Huzyg	Pass	Pending review	
Scenario 1	Change display name	4	Click Save	Save is submitted and a confirmation appears	A confirmation appeared (content checked in step 5); a console	Pass	Pending review	

Figure 27. The first page of a PDF report.

Reports use your final status and show Claude's next to it. A bug is left out when you reviewed its step as anything other than Fail; unreviewed bugs are included and labelled. Test passwords never appear; the account email may.

14 Tools

14.1 Search everything

Tools → *Search everything* (`Ctrl` + `Shift` + `F`) searches scenarios, steps, bugs, site notes and the library across all sessions. Double-click a result to open its session, scenario and step.

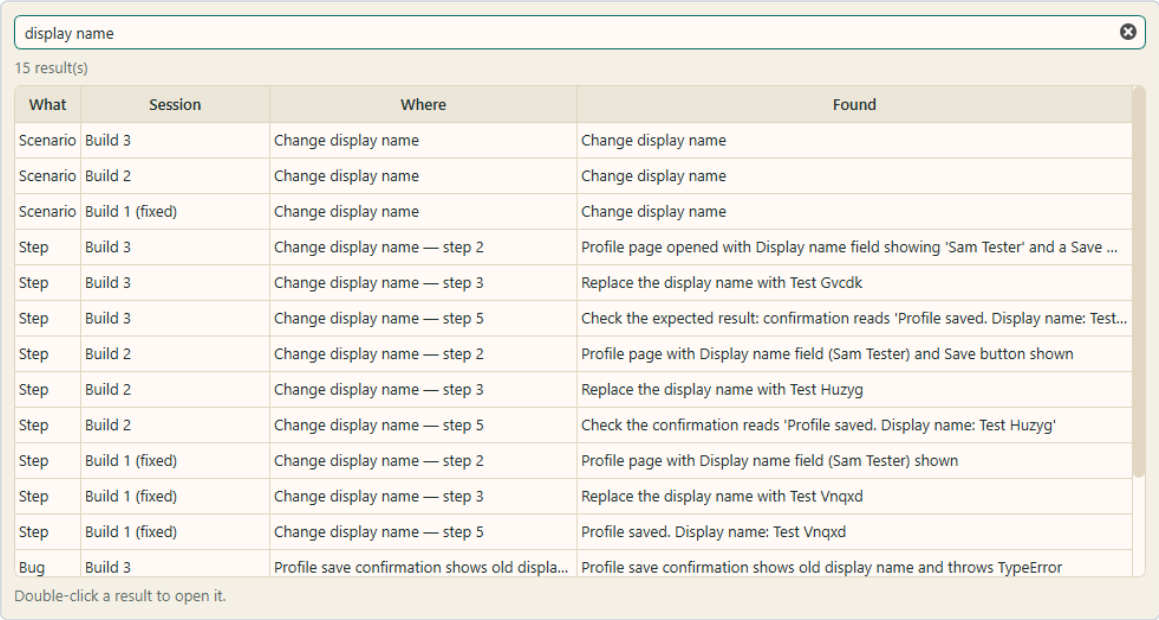


Figure 28. "display name" across three build sessions.

14.2 Trends

For each site: the pass rate of runs by day and the open bugs per session (a bug is open while the step's status that counts is Fail).

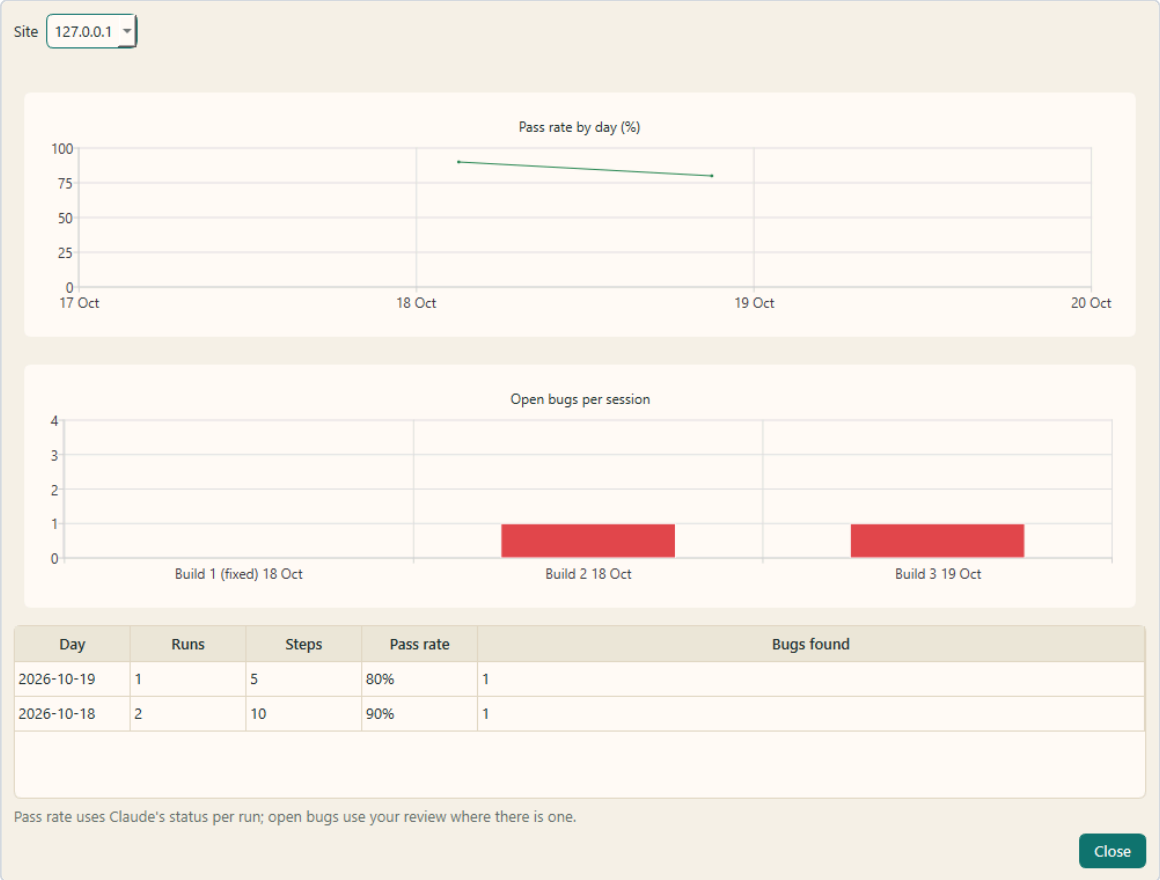


Figure 29. Pass rate by day, open bugs per build, and the numbers behind them.

14.3 Clean up and back up

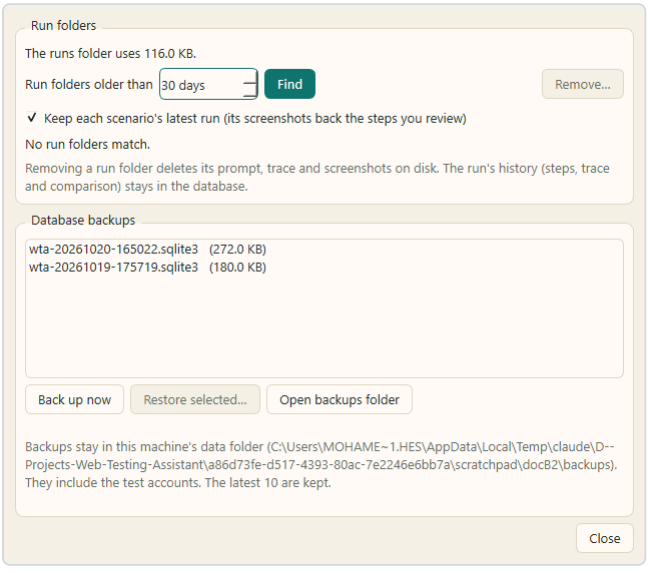


Figure 30. Disk use, old run folders and database backups.

- **Run folders:** see how much disk the runs folder uses and remove folders older than a number of days. Each scenario's latest run is kept by default (its screenshots back the steps you review); the run's history stays in the database.
- **Backups: Back up now** copies the database into the data folder on this computer (the latest 10 are kept). **Restore selected** takes effect at the next start, after a safety backup of the current database.

> **Backups stay on this computer**
> They contain the test accounts, which must not leave this machine.

15 Settings

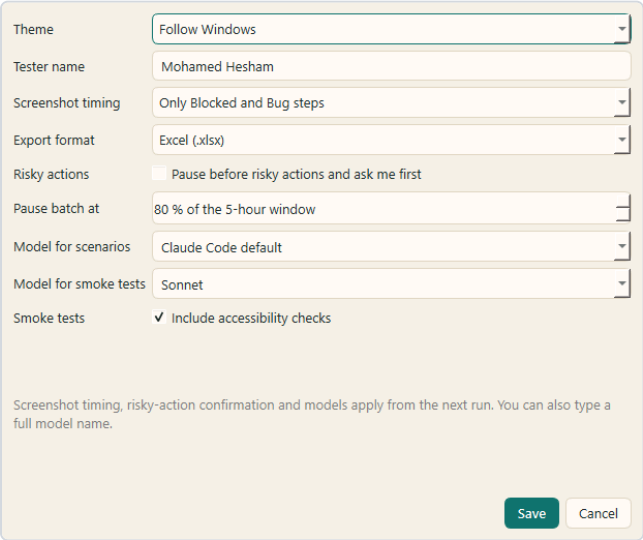


Figure 31. Settings on the toolbar or in Tools (`Ctrl` + `,`).

Setting	Default
Theme	Follow Windows (or Light: cream and teal; Dark)
Tester name	Empty; shown as Tester in reports
Screenshot timing	Only Blocked and Bug steps (or every step, or once per smoke page)
Export format	Excel
Risky actions	Off: no pause before risky actions
Pause batch at	80% of the 5-hour window (or Never)
Model for scenarios	Claude Code default
Model for smoke tests	Sonnet
Smoke tests	Include accessibility checks: on

Models accept the aliases Sonnet, Opus and Haiku or a full model name. Changes apply from the next run. Test Case Design has its own settings (section 6.5); the tester name here is also used in its exports.

16 Security and privacy

An AI that clicks around your application needs firm limits. These are built in.

Protection	How
Limited tools	Claude Code gets the Playwright browser tools and file access inside the run folder only. Running JavaScript in the page, closing the browser and installing browsers are denied; uploads only with an upload folder. No shell.
Your settings untouched	Claude Code runs restricted: it does not load your own Claude Code settings, hooks or MCP servers. The app never edits `~/.claude/settings.json`.
Allowed domains	The browser refuses pages outside the session's domains during a run; links to other domains are marked Blocked.
Page content is data	Text on a page that tries to instruct the AI is ignored and reported to you as a notice.
Test passwords	Stored only in the local database. Removed from prompts on disk, traces, results, exported reports and, after every run, from Claude Code's own saved sessions.
Your subscription	No API key is used or stored; `ANTHROPIC_API_KEY` is removed from Claude's environment. Claude credentials are never read.
Processes	Stop and Close all browsers end only processes the app started, by process ID, never by program name. Your own browsers are never touched.
Test Case Design	Claude Code gets no tools at all. Only the backlog being generated (its text, its pictures and your notes) is sent; workbooks, test cases and checklist runs stay in the local database.
Notifications	To show its name and logo, the app registers them for your Windows user (`HKCU\Software\Classes\AppUserModelId\WebTestingAssistant.App`); uninstalling removes the entry.
Local data	Sessions, results and backups stay on this computer. Nothing is sent anywhere except what Claude Code sends to Anthropic to run the test.

17 Plan usage and cost control

Tests run on your Claude subscription; the app helps you spend it wisely.

- **Usage gauges:** the top strip shows the 5-hour and weekly usage as Claude Code reports it during runs and generations. **Refresh** asks for a reading now with one tiny Haiku call.
- **Estimate:** next to Run batch, the expected share of the 5-hour window for the scenarios that would run, from your recent finished runs. Other Claude Code use at the same time counts too, so treat it as a guide.
- **Pause:** a batch pauses before the next scenario at the limit you set (80% by default) and asks whether to go on; a scenario already running is never interrupted. If usage is already over the limit, you are asked before the batch starts.
- **Models:** smoke tests use Sonnet by default; scenarios use Claude Code's default model; Test Case Design uses the latest Opus (or a pinned model), one call per backlog.
- **Sign in once:** the browser keeps its sign-in between scenarios, so Claude skips logging in again.
- **Free checks:** console errors, failed requests, load times, contrast and the marked screenshots are done by the app, not by Claude.

18 Data and files

Everything lives in `%APPDATA%\Web Testing Assistant` (set `WTA_DATA_DIR` to use another folder).

Path	Contents
`wta.sqlite3`	Sessions, accounts, scenarios, steps, reviews, bugs, runs, findings, site notes, library; workbooks, backlogs, pictures, test cases, generations and checklist runs
`runs\`	One folder per scenario run (below)
`designer-runs\`	One folder per test case generation (prompt and Claude Code output)
`checklists\`	Screenshots attached to checklist results
`exports\`	Default place for exported reports and test cases
`backups\`	Database backups (latest 10)
`browser-profile\`	The built-in browser's own cookies and cache, separate from your browsers
`settings.json`, `ui-state.ini`	Settings; window layout and the last open session
`usage.json`, `processes.json`	Last plan usage readings; processes the app started

Inside a run folder

File	Contents
`prompt.md`	The prompt given to Claude, with the test password replaced
`mcp.json`	Playwright MCP settings (attach to the built-in browser)
`trace.jsonl`	Claude Code's full output stream, password removed
`results.json`	The steps, statuses, bugs and notices Claude wrote
`screenshots\`	Claude's screenshots and the app's marked copies (`step-N-highlight.png`)
`fixtures\`, `print-N.pdf`	Copied upload files; printouts saved during the run

19 Troubleshooting

Problem	What to do
Run batch is disabled	Hover over it to see why. Usually Claude Code is not signed in or Node.js is missing (*Help → Check setup*), no session is open, or the browser's debugging port is not ready yet.
"Browser debugging port" warning	The port was taken when the app started. Restart the app.
Run fails with "Failed to refresh OAuth token"	Another Claude Code window was refreshing its sign-in at the same moment. Wait a minute and run again.
Usage shows "–" or "needs refresh"	No reading yet, or the window has reset since the last one. Click the refresh button (↻) or start a run.
Generate test cases is disabled or refused	Claude Code must be installed and signed in (*Help → Check setup*); a backlog with no text and no pictures needs a note first.
No Windows notifications	Check *Windows Settings → System → Notifications*: notifications must be on for Web Testing Assistant, and Do not disturb off.
A step is Blocked "outside the allowed domains"	Add the domain to the session (**Details** on the toolbar) if the test should go there.
A batch paused	Your 5-hour usage reached the pause limit. Continue anyway, or stop and run the rest after the window resets.
SmartScreen blocks the installer	Choose **More info → Run anyway**: the installer is not code-signed.
Check an installation	Run `WebTestingAssistant.exe --self-check report.json`: it checks the browser engine, icons, charts and exports, and reports Node.js and Claude Code, without opening the window.

20 For developers

20.1 Running from source

```
python -m venv .venv
.venv\Scripts\python -m pip install -e .
.venv\Scripts\python -m wta
```

Python 3.12; dependencies: PySide6, SQLAlchemy, openpyxl, python-docx, jsonschema. The project rules are in `CLAUDE.md` and the build history and findings in `PLAN.md`.

20.2 Code map

Module (src/wta)	Responsibility
`app.py` · `launchpad.py`	Main window, modes, menus, toolbar, running batches; the Launchpad
`designer/`	Test Case Design: workbook analysis, prompt builder and versioned prompts, generation, validation, workspace, export, hand-over
`checklist_tab.py` · `db/checklists.py`	Manual checklist runs
`usage.py` · `usage_bar.py` · `notify.py`	Plan usage readings and gauges; Windows notifications under the app's identity
`claude_launcher.py`	Everything about starting Claude Code: arguments, tools, environment, sign-in and Node.js checks
`runner.py`	One scenario run and a batch: process, stream, results, pause, history
`prompt.py` · `results.py` · `stream.py`	The prompt; reading results.json; parsing Claude Code's stream
`browser.py` · `domain_guard.py` · `cdp.py`	Built-in browser, pop-ups, banner; allowed-domain guard; debugging port
`page_monitor.py` · `accessibility.py` · `highlight.py`	The app's own page checks and marked screenshots
`db/`	Models, sessions, scenarios, runs, library, search, site notes
`export/`	Report model and the Excel, Word, PDF and Markdown writers
`session_files.py`	Removing test passwords from Claude Code's saved sessions

20.3 Tests and test pages

```
.venv\Scripts\python -m unittest discover -s tests
```

The tests use a fake `claude` and never use plan. Local pages for trying the app are in `test_pages/` (a login app with a deliberate bug, a prompt-injection page, an upload page, a printable invoice); serve them with

`.venv\Scripts\python -m http.server 8765 --bind 127.0.0.1 --directory test_pages`.

20.4 Building the installer and this document

```
.venv\Scripts\python -m pip install -r tools\build-requirements.txt
.venv\Scripts\python tools\build_installer.py
.venv\Scripts\python tools\build_docs.py
```

`build_installer.py` bundles the app with PyInstaller (the exe carries its name and version), runs the bundle's self-check and wraps it with Inno Setup 6 into `dist\installer\WebTestingAssistant-Setup-<version>.exe`. The version comes from `src\wta\__init__.py`. `build_docs.py` prints `docs\source\documentation.html` to `docs\Web-Testing-Assistant-Documentation.pdf`.

20.5 Versions and releases

Every branch carries a version `A.B.C` in `src\wta\__init__.py` and `pyproject.toml` (both the same):

Number	Counts	Changes when
A	merges of `pre-release` into `main`	`pre-release` is merged into `main`: A + 1
B	merges of a branch into `pre-release`	a branch is merged into `pre-release`: B + 1, C back to 1
C	branches created from `pre-release` since its last merge	the n-th new branch gets C = n + 1

`pre-release` itself always ends in `.1`. Example from `pre-release` 1.1.1: the first new branches are 1.1.2 and 1.1.3; merging one makes `pre-release` 1.2.1; merging `pre-release` into `main` then makes both 2.2.1. On GitHub, builds of branches and of `pre-release` are published as **pre-releases** tagged `v<A.B.C>`; only builds of `main` are full releases.

A Keyboard shortcuts

Keys	Action
`Alt` + `M` · `Ctrl` + `L`	Mode menu · Back to Launchpad
`Ctrl` + `Q`	Quit
`Ctrl` + `N` · `Ctrl` + `O`	New session · Open session (Regression, Smoke)
`F5` · `Shift` + `F5`	Run batch · Stop
`Ctrl` + `E`	Export report
`Ctrl` + `,`	Settings
`Ctrl` + `Shift` + `F`	Search everything
`A` · `O` · `R`	Approve · Override · Request re-run (review)
`↑` · `↓`	Previous · next step

B Glossary

Term	Meaning
Launchpad	The start screen where you choose a mode.
Mode	Test Case Design, Regression Testing or Smoke Testing.
Workbook	An uploaded Excel file of product backlogs, with its versions (v1, v2 …).
Backlog	One numbered item of a workbook: its text and pictures.
Product context	A workbook's description of the application under test, used in the prompt.
Output preset	The format of generated test cases: fields, ID pattern, priority labels, steps style, title prefix.
Checklist run	Designed test cases that you run by hand, recording Pass, Fail, Blocked or Skipped.
Pre-release	A build of a branch or of `pre-release`, published on GitHub for trying out.
Session	One testing assignment: an application, its accounts and allowed domains, and its batches.
Batch	Up to five scenarios (or smoke pages) run together.
Scenario	A test written as steps and an expected result.
Smoke test	A broad check of one page; its checks are written by Claude.
Run	One Claude Code run of one scenario, with its own folder and history.
Final status	The status set by your review; the one reports use.

Site note	Your instruction for every following run on a site.
Library item	A saved scenario or smoke page, reusable across sessions.
Claude Code	Anthropic's command-line agent, run by the app with your subscription.
Playwright MCP	The browser toolkit Claude Code uses to drive the built-in browser.
CDP	Chrome DevTools Protocol: the local debugging connection to the built-in browser.